

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - ICT

Bacharelado em Ciência e Tecnologia / Engenharia de Computação

Predição de Falhas em Turbinas Eólicas Utilizando Modelos de Aprendizado Profundo e Dados Exógenos

Relatório Final de Iniciação Científica submetido à
Fundação de Amparo à Pesquisa do Estado de São Paulo (FAPESP).

Aluno: Thiago Solé Gomes Heleno

Orientador: Marcos G. Quiles

Coorientador: Mateus Giesbrecht (FEEC/Unicamp)

Vigência da Solicitação: 01/06/2025 – 31/05/2026

São José dos Campos, maio de 2026

Resumo

A energia eólica é peça-chave na transição para uma matriz energética limpa, mas sua viabilidade depende da operação contínua das turbinas. Falhas mecânicas e eletrônicas geram custos elevados de O&M e perda de geração, motivando o uso de manutenção preditiva apoiada em dados SCADA (Supervisory Control and Data Acquisition). O presente projeto de Iniciação Científica investigou modelos de aprendizado profundo para detecção de anomalias em turbinas eólicas a partir do dataset público CARE_To_Compare (Wind Farm C, 3,18 M amostras de 10 minutos, 957 colunas (incluindo identificadores), razão Normal:Anomalia $\approx 51:1$). O escopo original previa integração com dados meteorológicos exógenos (CPTEC/INPE, OpenWeather-Map), mas a anonimização total das coordenadas e datas absolutas no dataset inviabilizou essa fusão; o projeto foi readequado para comparar dois paradigmas alternativos — supervisionado e não supervisionado — e para explorar *data augmentation* via injeção de falhas sintéticas. Foram implementados cinco notebooks: (i) CNN, LSTM e CNN-LSTM supervisionados (replicação de Qi et al., 2024); (ii) Autoencoder CNN-BiLSTM com camada de atenção em PyTorch; (iii) Autoencoder MLP em Keras com otimização Optuna e métrica CARE; (iv) classificador MLP de 3 classes com falhas induzidas; e (v) Autoencoder MLP com calibração de threshold por falhas induzidas. Os autoencoders não supervisionados atingiram AUC-ROC de até 0,872 (vs. 0,66 do melhor supervisionado), confirmando a vantagem do paradigma sob desbalanceamento severo, embora a precisão amostral baixa ($\sim 2\text{--}3\%$) ainda inviabilize uso direto em produção. O CARE Score do NB3 (0,699) e a calibração induzida do NB5 (CARE 0,545 com Acc por evento de 0,98) sugerem *ensembles* como linha futura natural.

Palavras-chave: Turbinas Eólicas. Manutenção Preditiva. Aprendizado Profundo. Detecção de Anomalias. Autoencoder. SCADA. CARE Score.

Sumário

1	Introdução	10
2	Justificativa	11
3	Objetivo	12
3.1	Aprendizado	12
3.1.1	Fundamentos de Aprendizado de Máquina	12
3.1.2	Redes Neurais Artificiais	12
3.1.3	Aprendizado Profundo	12
3.2	Treinamento	13
3.2.1	Pipeline de Treinamento	13
3.2.2	Datasets de Aprendizado	13
4	Fundamentação Teórica	13
4.1	Manutenção Preditiva em Turbinas Eólicas	13
4.2	Sistemas SCADA	14
4.3	Séries Temporais e Detecção de Anomalias	14
4.4	Arquiteturas de Aprendizado Profundo	14
4.4.1	LSTM	14
4.4.2	CNN1D	15
4.4.3	MLP (<i>Multi-Layer Perceptron</i>)	16
4.4.4	Mecanismo de Atenção (<i>MultiHeadAttention</i>)	16
4.5	Métricas de Avaliação	17
4.5.1	Métricas Padrão de Classificação	17
4.5.2	CARE Score (framework adotado)	17
5	Metodologias	18
5.1	Bases de Dados	18
5.1.1	Repositório SCADA Público	18
5.1.2	Dados Meteorológicos	19
5.1.3	Análise Exploratória dos Dados (EDA)	20
5.2	Modelos e Notebooks Implementados	20

5.2.1	NB1 — CNN-LSTM (Supervisionado, replicação Qi et al. 2024)	20
5.2.2	NB2 — CNN-BiLSTM-Attention Autoencoder (Não supervisionado, PyTorch)	22
5.2.3	NB3 — Dense MLP Autoencoder (Não supervisionado, Keras + Optuna)	23
5.2.4	NB4 — Classifier MLP 3-class com Falhas Induzidas (Supervisionado)	23
5.2.5	NB5 — Autoencoder MLP com Calibração por Falhas Induzidas (Não supervisionado)	23
5.3	Data Processing	24
5.3.1	Extração e Conversão	24
5.3.2	Limpeza de Dados e Tratamento de Outliers	24
5.3.3	Imputação de Dados Faltantes	25
5.3.4	Sincronização Temporal	25
5.3.5	Seleção de Variáveis (Feature Selection)	25
5.3.6	Normalização e Escalonamento	25
5.4	Hiperparâmetros e Otimização	26
5.5	Estratégia de Validação	26
5.6	Reprodutibilidade	26
6	Experimentos	27
6.1	Configuração Experimental	27
6.2	Experimentos sem Dados Exógenos	27
6.3	Experimentos com Dados Exógenos	27
6.4	Ablation Study	28
6.5	Custo Computacional	28
7	Problemas com Dados	29
7.1	Escassez de Dados de Anomalia	29
7.2	Localização Não Divulgada dos Sítios	29
7.3	Impossibilidade de Uso de Dados Meteorológicos Externos	29
7.4	Qualidade dos Dados SCADA	30
8	Problemas nos Modelos	31
8.1	Desbalanceamento de Classes	31
8.2	Overfitting	31

8.3	Custo Computacional do Treinamento	31
8.4	Generalização para Diferentes Turbinas	32
8.5	Dificuldades de Implementação	32
9	Comparação com Outras Pesquisas	32
9.1	Trabalhos Relacionados na Literatura	32
9.2	Análise Comparativa de Resultados	33
10	Resultados Obtidos	33
10.1	Resultados Quantitativos	33
10.2	Resultados Qualitativos	34
10.3	Discussão dos Resultados	38
11	Limitações do Estudo e Atribuições	41
11.1	Desvios em Relação ao Plano Original	41
11.2	Componentes Replicados de Fontes Externas	41
11.3	Outras Limitações	41
12	Cronograma Realizado vs Planejado	42
13	Matérias Cursadas na Faculdade	42
14	Lições Aprendidas	42
15	Trabalhos Futuros	43
16	Considerações Finais	44

Lista de Figuras

1	Diagrama da célula LSTM com os três <i>gates</i> (forget, input, output). Fonte: dida.do, “Long Short-Term Memory Networks (LSTM): subtleties of mathematical inference”, https://dida.do/blog/long-short-term-memory-network-lstm-subtleties-of-mathematical-inference	15
2	Diagrama esquemático de uma rede convolucional (extração de <i>features</i> via <i>convolution+pooling</i> seguida de classificação por camadas <i>fully connected</i>). Fonte: Phung and Rhee (2019).	15
3	Topologia genérica de um <i>Multi-Layer Perceptron</i> (camada de entrada, camadas ocultas e camada de saída totalmente conectadas), arquitetura base do encoder/decoder dos autoencoders MLP utilizados nos NB3 e NB5. Fonte: Afan et al. (2021).	16
4	Arquitetura Transformer completa (<i>encoder/decoder</i> com <i>multi-head attention</i> e <i>positional encoding</i>), com zoom em <i>multi-head attention</i> e <i>scaled dot-product attention</i> . Apenas a camada <code>MultiHeadAttention</code> foi utilizada neste projeto, dentro do encoder do NB2. Fonte: adaptado de Vaswani et al. (2017).	17
5	Desbalanceamento de classes no Wind Farm C considerando <code>status_type_id</code> : estados 0 (Normal) + 2 (Idling) vs. demais.	20
6	Série temporal de potência gerada (exemplo de turbina, primeiras 2000 amostras).	21
7	Histograma de velocidade do vento (<code>wind_speed_236_avg</code> , amostra de 40k registros).	21
8	Matriz de correlação para subconjunto de variáveis SCADA (colunas <code>_avg</code>).	22
9	Boxplots antes e depois do <i>clipping</i> Q0,1%–Q99,9% por feature.	24
10	Mapa de calor de sensores inativos (fração de valores 0 ou NaN) por dataset × feature SCADA. No <code>CARE_To_Compare</code> , sensores offline aparecem zerados (não NaN); colunas claras indicam instrumentação ausente em todos os datasets, enquanto pixels claros isolados marcam quedas pontuais.	30
11	Comparativo de F1-Score (amostra) entre todas as configurações dos cinco notebooks.	35
12	Comparativo de AUC-ROC: autoencoders não supervisionados (NB2/NB3) superam os modelos supervisionados do NB1.	35
13	Curvas ROC aproximadas (modelo binormal a partir do AUC reportado).	36

14	Matrizes de confusão reconstruídas a partir de <i>precision/recall</i> e do tamanho do conjunto de teste.	36
15	CARE Score e subcomponentes $EF1_2$, Acc por evento e WS.	37
16	NB5 — trade-off <i>precision/recall</i> entre os três thresholds.	37
17	NB3 vs NB5 ocupam pontos opostos do trade-off <i>precision/recall</i>	38
18	Ranking de importância das <i>features</i> produzido pelo XGBoost (NB2).	38
19	Análise por feature (AUROC do erro de reconstrução — NB2).	39
20	Curvas ROC e Precision-Recall do NB2.	39
21	NB4 — comparativo Modelo Induzido vs Baseline (loss, matrizes, ROC, PR, CARE).	39
22	NB5 — loss do AE, distribuição RMSE, série temporal e CARE para os três thresholds.	40

Lista de Tabelas

1	Visão geral dos cinco notebooks implementados.	21
2	Hiperparâmetros finais por notebook.	26
3	Ablação pós-treino do NB2: impacto de cada estratégia v4.	28
4	Custo computacional estimado por <i>notebook</i> (GPU NVIDIA RTX A6000, 48 GB).	29
5	Métricas consolidadas por notebook/configuração no conjunto de teste.	34
6	NB3 vs NB5 — pontos opostos do trade-off precision/recall.	34
7	Cronograma realizado vs. planejado (A1–A9 do plano FAPESP).	42
8	Disciplinas cursadas durante a vigência da IC (2025.2 e 2026.1).	43

Lista de Abreviaturas e Siglas

AE	Autoencoder
AMP	Automatic Mixed Precision
AUC-PR	Area Under the Precision-Recall Curve
AUC-ROC	Area Under the Receiver Operating Characteristic Curve
BiLSTM	Bidirectional Long Short-Term Memory
CARE	Composite Anomaly Reliability Evaluation
CNN	Convolutional Neural Network
CNN1D	Convolutional Neural Network 1D (temporal)
DL	Deep Learning (Aprendizado Profundo)
EDA	Exploratory Data Analysis (Análise Exploratória de Dados)
FN	False Negative
FP	False Positive
GPU	Graphics Processing Unit
IC	Iniciação Científica
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
ML	Machine Learning (Aprendizado de Máquina)
MLP	Multi-Layer Perceptron
MSE	Mean Squared Error
NB	Notebook
O&M	Operação e Manutenção
PReLU	Parametric Rectified Linear Unit
ReLU	Rectified Linear Unit
RMSE	Root Mean Squared Error
RNN	Recurrent Neural Network
SCADA	Supervisory Control and Data Acquisition
SELU	Scaled Exponential Linear Unit
TN	True Negative
TP	True Positive
TPU	Tensor Processing Unit
WS	Warning Score

1 Introdução

O aprendizado de máquina (ML) vem ganhando destaque como ferramenta essencial para melhorar a confiabilidade e a eficiência de sistemas industriais complexos, como é o caso das turbinas eólicas, trazendo novas perspectivas para as estratégias de manutenção preditiva. Modelos de ML conseguem analisar padrões temporais em dados operacionais (vibração, temperatura, rotação, etc.) e integrar variáveis externas, prevendo assim falhas com maior precisão. Indo além da simples detecção de anomalias, técnicas avançadas como as redes neurais profundas possibilitam modelar as interações dinâmicas entre componentes mecânicos e fatores ambientais, o que ajuda a antecipar cenários críticos que poderiam levar a paradas custosas. Essas abordagens não apenas diminuem custos operacionais, mas também favorecem a sustentabilidade energética ao minimizar desperdícios.

Contudo, construir modelos preditivos robustos esbarra em desafios importantes ligados à disponibilidade e qualidade dos dados. Um dos maiores obstáculos é a escassez de bases públicas de dados SCADA extensas e bem rotuladas. Além disso, os conjuntos de dados disponíveis sofrem com um desbalanceamento severo: as falhas são eventos extremamente raros em comparação aos longos períodos de operação normal. Essa falta de amostras representativas de anomalias dificulta significativamente o treinamento, pois impede que os modelos de Redes Neurais aprendam adequadamente os padrões de falha, que por sua natureza já são altamente complexos e variáveis. A dificuldade de capturar informações em condições operacionais extremas acaba limitando a capacidade do modelo de generalizar eventos raros, comprometendo a escalabilidade e a confiabilidade das soluções propostas.

Inicialmente, este projeto propunha o desenvolvimento de modelos preditivos combinando os dados operacionais da turbina com variáveis meteorológicas externas. Contudo, durante a execução da pesquisa, esbarrou-se em uma limitação técnica intrínseca aos dados abertos: os repositórios públicos anonimizam completamente a localização geográfica dos parques eólicos (fornecendo apenas identificadores genéricos) por questões de confidencialidade comercial. Sem as coordenadas exatas e datas absolutas, tornou-se impossível alinhar espacialmente e temporalmente os dados SCADA com bases meteorológicas externas (como CPTEC/INPE e OpenWeatherMap).

Diante dessa inviabilidade, o escopo da pesquisa foi readequado. Em vez de comparar modelos com e sem dados exógenos, o projeto passou a focar na comparação entre diferentes paradigmas de detecção — supervisionado versus não supervisionado — e na exploração

de técnicas alternativas para contornar a escassez de dados, como o uso de data augmentation via injeção de falhas sintéticas (induzidas). Como resultado, foram investigadas arquiteturas como CNN1D, LSTM e Autoencoders com mecanismos de atenção, e estabelecidos protocolos de avaliação para identificar a abordagem mais viável para o monitoramento de turbinas em cenários de dados escassos e desbalanceados.

2 Justificativa

Antecipar falhas em turbinas eólicas é fundamental para diminuir os custos com manutenções corretivas e evitar interrupções na geração de energia. A princípio, integrar dados meteorológicos aos modelos preditivos ajudaria a identificar padrões mais complexos, como o efeito de rajadas de vento ou de variações de temperatura no comportamento da máquina. No entanto, a realidade técnica dos repositórios de dados abertos — que ocultam a localização dos parques e carecem de amostras de falhas bem documentadas — evidencia uma lacuna crítica na pesquisa aplicada: a urgência de se criar modelos robustos que operem com alta precisão mesmo sob a escassez de dados anômalos e sem o apoio de variáveis climáticas externas.

Nesse contexto, o projeto justifica-se fortemente pela necessidade de desenvolver soluções metodológicas que contornem essa restrição. Ao redirecionar o foco para a comparação entre métodos supervisionados e não supervisionados, e ao explorar técnicas como o data augmentation via injeção de falhas sintéticas, o trabalho busca aumentar a capacidade de generalização dos modelos de redes neurais quando submetidos a dados severamente desbalanceados.

Além de sua contribuição técnica, o uso de bases de dados públicas e de ferramentas de código aberto (como TensorFlow e PyTorch) favorece a acessibilidade e a reprodutibilidade do projeto, um ponto importante frente às demandas por transparência e sustentabilidade no setor energético. Por fim, a pesquisa mantém sua forte contribuição para o debate acadêmico sobre a aplicação do aprendizado de máquina industrial, trazendo insights práticos para aplicações em ambientes dinâmicos e propondo caminhos para viabilizar a manutenção preditiva diante de limitações reais na disponibilidade de dados.

3 Objetivo

3.1 Aprendizado

A vertente formativa deste projeto visou capacitar o aluno nos fundamentos teóricos e práticos necessários para a investigação proposta, com ênfase em aprendizado de máquina aplicado a séries temporais e em ferramentas computacionais para análise de dados SCADA. Esse eixo se desenvolveu em paralelo às atividades experimentais, com acompanhamento integral do **curso de Redes Neurais Artificiais ministrado pelo orientador Prof. Dr. Marcos G. Quiles¹**, cujas vídeo-aulas e materiais de apoio estão organizados localmente em `aprendizado_curso/`. Foram conduzidos exercícios práticos em PyTorch, NumPy e Pandas, replicando os blocos do curso antes de integrá-los aos pipelines do projeto.

3.1.1 Fundamentos de Aprendizado de Máquina

Estudo dos conceitos básicos de aprendizado supervisionado e não supervisionado: regressão, classificação binária e multi-classe, função de perda (MSE, MAE, Cross-Entropy), regularização (L_1 , L_2 , *dropout*), divisão treino/validação/teste, validação cruzada e principais métricas de avaliação (acurácia, precisão, recall, F1, AUC-ROC, RMSE, MAE).

3.1.2 Redes Neurais Artificiais

Estudo do *perceptron* simples, MLP (*Multi-Layer Perceptron*), funções de ativação (ReLU, PReLU, SELU, Sigmoid, Softmax), *backpropagation* via regra da cadeia, otimizadores baseados em gradiente (SGD, Adam, AdamW) e estratégias de inicialização (He, Xavier).

3.1.3 Aprendizado Profundo

Aprofundamento em redes recorrentes (RNN, LSTM, BiLSTM e seus *gates* de *input*, *forget* e *output*), redes convolucionais 1D (kernel, *stride*, *padding*, *pooling*), mecanismos de atenção (*scaled dot-product attention*, *multi-head attention*), autoencoders e suas variantes (denso, convolucional, recorrente). O conteúdo foi acompanhado por exercícios práticos em PyTorch reproduzindo os blocos antes de integrá-los aos pipelines do projeto.

¹Playlist disponível em: <https://www.youtube.com/watch?v=kzFqGhK8Q2s&list=PLxI8Can9yAHfB2om9dqhsrsjRAf-YJaro>.

3.2 Treinamento

A etapa de treinamento contemplou tanto experimentação inicial em datasets reduzidos quanto a execução final dos cinco *notebooks* sobre o Wind Farm C completo.

3.2.1 Pipeline de Treinamento

O pipeline padrão adotado foi: (1) carregamento dos CSVs SCADA; (2) divisão treino/validação/teste (por evento no NB1 e NB4; temporal 70/15/15 dinâmico no NB2; coluna `train_test` do CARE com sub-split 80/20 interno no NB3 e NB5 — detalhes na Seção 5.5); (3) pré-processamento (*clipping* de outliers, imputação, normalização); (4) seleção de *features*; (5) construção de janelas temporais quando aplicável; (6) treinamento com *early stopping* sobre *val_loss*; (7) calibração de *threshold* sobre validação; (8) avaliação no teste.

3.2.2 Datasets de Aprendizado

Antes da execução final no Wind Farm C, foram utilizados subconjuntos pequenos para validar a corretude dos pipelines e para fins didáticos: amostras balanceadas geradas a partir de poucos CSVs e datasets sintéticos de classificação binária do scikit-learn (`make_classification`, `make_moons`) para depurar funções de perda, otimizadores e callbacks de *early stopping*.

4 Fundamentação Teórica

4.1 Manutenção Preditiva em Turbinas Eólicas

A manutenção preditiva (*Predictive Maintenance* — *PdM*) busca antecipar falhas a partir do monitoramento contínuo da condição do equipamento, em contraste com a manutenção corretiva (após a falha) e a preventiva (em janelas fixas, independentes do estado real). Em turbinas eólicas, o custo das paradas não programadas é elevado, especialmente em parques *offshore*, e a degradação dos componentes (caixa multiplicadora, rolamentos, gerador, sistema de *pitch*) costuma evoluir gradualmente, deixando assinaturas detectáveis em sensores de vibração, temperatura e potência elétrica antes da falha. Esse cenário motiva a aplicação de técnicas de detecção de anomalias e prognóstico em séries temporais.

4.2 Sistemas SCADA

O sistema SCADA (*Supervisory Control and Data Acquisition*) é a infraestrutura de monitoramento padrão de parques eólicos modernos, agregando dezenas a centenas de sensores instalados ao longo da turbina (nacele, rotor, hub, conversor, transformador). Os dados são tipicamente registrados em janelas de 10 minutos com agregações estatísticas (média, máximo, mínimo, desvio-padrão por janela), gerando um conjunto rico de variáveis (no Wind Farm C, 238 sensores físicos $\times$ 4 estatísticas = 957 colunas (incluindo identificadores)). Apesar de ser uma instrumentação menos precisa que sistemas dedicados de monitoramento de condição, sua disponibilidade ampla e baixo custo marginal tornam-no atrativo para algoritmos de aprendizado de máquina.

4.3 Séries Temporais e Detecção de Anomalias

Uma série temporal é uma sequência $\{x_t\}_{t=1}^T$ indexada pelo tempo. Em turbinas eólicas, cada sensor produz uma série multivariada com periodicidades (diurna, sazonal) e regimes operacionais (partida, geração nominal, parada). A detecção de anomalias busca identificar pontos ou subsequências cujo comportamento desvia significativamente do padrão observado. As três famílias principais de abordagens são: (i) baseadas em *thresholds* estatísticos (Z-score, IQR); (ii) supervisionadas, treinando classificadores binários ou multi-classe sobre exemplos rotulados; (iii) não supervisionadas, treinando modelos para reconstruir ou prever o comportamento normal, e usando o erro de reconstrução/predição como *score* de anomalia.

4.4 Arquiteturas de Aprendizado Profundo

4.4.1 LSTM

A LSTM (*Long Short-Term Memory*) é uma RNN projetada para mitigar os problemas de gradiente desaparecente das RNNs tradicionais. Cada célula mantém um estado interno C_t controlado por três *gates*: *forget gate* $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$, *input gate* $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$ e *output gate* $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$. O estado é atualizado por $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$, onde $\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$, e a saída é $h_t = o_t \odot \tanh(C_t)$. A versão BiLSTM processa a sequência em ambas as direções e concatena os estados, capturando contexto passado e futuro. A Figura 1 ilustra a estrutura interna da célula.

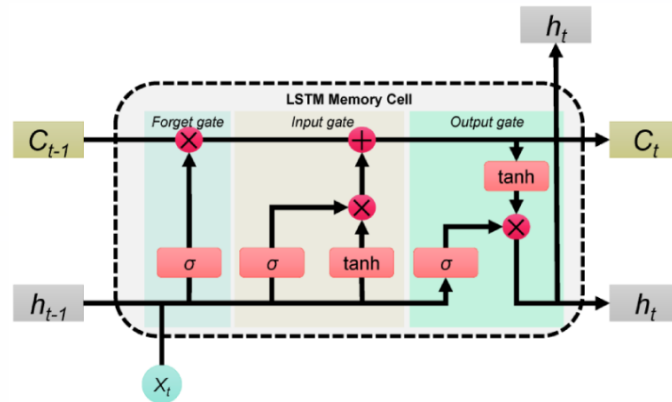


Figura 1: Diagrama da célula LSTM com os três *gates* (forget, input, output). Fonte: dida.do, “Long Short-Term Memory Networks (LSTM): subtleties of mathematical inference”, <https://dida.do/blog/long-short-term-memory-network-lstm-subtleties-of-mathematical-inference>.

4.4.2 CNN1D

A CNN 1D aplica filtros convolucionais ao longo do eixo temporal: dado um sinal $x \in \mathbb{R}^{T \times C}$, um filtro $W \in \mathbb{R}^{k \times C}$ produz $y_t = \sum_{i=0}^{k-1} W_i x_{t+i} + b$. *Stride* controla o passo entre janelas e *padding* preserva o comprimento. Camadas de *pooling* (max, avg) reduzem dimensionalidade temporal, e o empilhamento de várias camadas convolucionais aumenta o campo receptivo, permitindo que o modelo capte padrões em diferentes escalas temporais. A Figura 2 mostra o diagrama de uma camada convolucional típica.

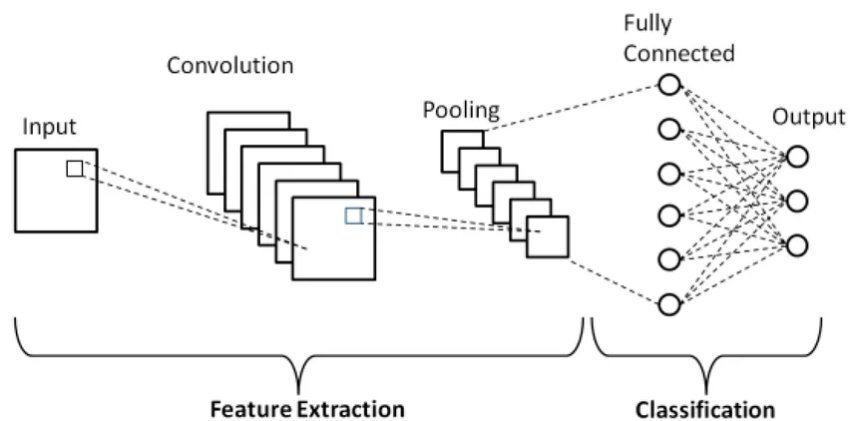


Figura 2: Diagrama esquemático de uma rede convolucional (extração de *features* via *convolution+pooling* seguida de classificação por camadas *fully connected*). Fonte: Phung and Rhee (2019).

4.4.3 MLP (*Multi-Layer Perceptron*)

O MLP é uma rede totalmente conectada composta por uma camada de entrada, uma ou mais camadas ocultas e uma camada de saída. Cada neurônio aplica a transformação afim $z = Wx + b$ seguida de uma função de ativação não linear (PReLU nos NB3, NB4 e NB5). Na configuração autoencoder (NB3 e NB5), o MLP é treinado para reconstruir a própria entrada: o *encoder* comprime o vetor de *features* até um *bottleneck* de dimensão reduzida (*code_size*), e o *decoder* simétrico reconstrói a entrada a partir dessa representação compacta. O erro de reconstrução (RMSE por amostra) é usado como *score* de anomalia — amostras fora do padrão normal aprendido exibem erro alto. A Figura 3 ilustra a topologia do autoencoder MLP.

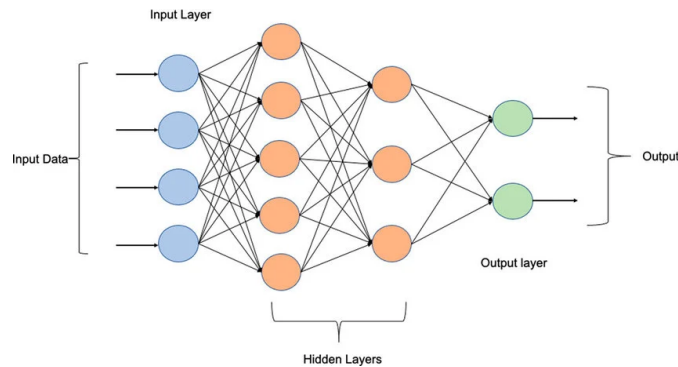


Figura 3: Topologia genérica de um *Multi-Layer Perceptron* (camada de entrada, camadas ocultas e camada de saída totalmente conectadas), arquitetura base do encoder/decoder dos autoencoders MLP utilizados nos NB3 e NB5. Fonte: Afan et al. (2021).

4.4.4 Mecanismo de Atenção (MultiHeadAttention)

A atenção de produto-escalar escalado calcula $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$, onde Q , K , V são projeções lineares da entrada (*queries*, *keys*, *values*). A versão *multi-head* executa h atenções paralelas em sub-espacos diferentes e concatena os resultados, permitindo que o modelo combine padrões em múltiplos pontos da sequência simultaneamente. **Importante:** no NB2 deste projeto, a atenção é utilizada como uma *camada* dentro de uma arquitetura CNN-BiLSTM, e **não** como Transformer completo (não há *positional encoding* explícito nem blocos *encoder/decoder* no padrão original de Vaswani et al. (2017)). Essa distinção é importante em relação ao plano original, que previa Transformers completos. A Figura 4 apresenta a arquitetura Transformer completa para referência conceitual.

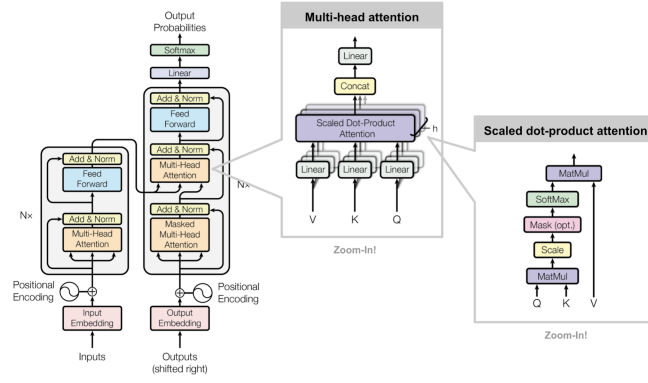


Figura 4: Arquitetura Transformer completa (*encoder/decoder com multi-head attention e positional encoding*), com zoom em *multi-head attention* e *scaled dot-product attention*. Apenas a camada MultiHeadAttention foi utilizada neste projeto, dentro do encoder do NB2. Fonte: adaptado de Vaswani et al. (2017).

4.5 Métricas de Avaliação

4.5.1 Métricas Padrão de Classificação

Sejam TP , TN , FP , FN as entradas da matriz de confusão. Define-se:

$$\text{Acuracia} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precisao} = \frac{TP}{TP + FP}, \quad (1)$$

$$\text{Revocacao} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot P \cdot R}{P + R}, \quad (2)$$

$$F_\beta = \frac{(1 + \beta^2) \cdot P \cdot R}{\beta^2 \cdot P + R}. \quad (3)$$

A AUC-ROC é a área sob a curva ROC ($TPR \times FPR$), independente de *threshold*, e mede a qualidade do ordenamento dos *scores*. Em datasets fortemente desbalanceados, a AUC-PR (área sob a curva Precision-Recall) é mais informativa que a AUC-ROC.

4.5.2 CARE Score (framework adotado)

O CARE Score (*Composite Anomaly Reliability Evaluation*) é uma métrica multi-objetivo composta por $F1_\beta$ e $EF1_\beta$ (*event-level F_β*), Acc por evento normal e WS (*Warning Score*, que penaliza alarmes tardios). A agregação final é:

$$\text{CARE} = \frac{F1_2 + WS + EF1_2 + 2 \cdot \text{Acc}}{5}, \quad (4)$$

onde $\beta = 2$ pondera o *recall* duas vezes mais que a *precision*, e Acc é a acurácia por evento normal (especificidade em nível de evento). A fórmula combina sensibilidade amostral ($F1_2$), ante-

cipação temporal (WS), sensibilidade por evento ($EF1_2$) e especificidade em eventos normais (Acc). **Atribuição:** o framework CARE Score e a análise ARCANA (importância de *features*) seguem a especificação do repositório público EnergyFaultDetector (AEFDI), disponível em <https://github.com/AEFDI/EnergyFaultDetector>. As funções (`care_score_formula`, `arcana_loss`, `run_arcana_for_sample`) foram *re-implementadas inline* no notebook NB3 baseando-se nessa especificação, mas a **conceituação metodológica não constitui contribuição original deste trabalho**. Foram utilizadas como ferramentas de avaliação padronizadas para permitir comparação com a literatura.

5 Metodologias

5.1 Bases de Dados

5.1.1 Repositório SCADA Público

A base utilizada foi o **CARE_To_Compare**, repositório público com 95 datasets cobrindo aproximadamente 89 anos de dados SCADA distribuídos em 36 turbinas de três parques eólicos: **Wind Farm A** (*onshore*, Portugal — 22 datasets baseados em dados abertos da EDP), **Wind Farm B** e **Wind Farm C** (ambos *offshore*, Alemanha, anonimizados). No total, 44 dos 95 datasets contêm um evento de anomalia rotulado e 51 representam comportamento normal — ou seja, o conjunto agregado é **balanceado em nível de evento**, embora cada dataset individualmente seja severamente desbalanceado em nível de amostra.

Cada dataset é fornecido como um único arquivo CSV contendo a série temporal de uma turbina (treino + predição), com amostragem de 10 minutos. O número de *features* varia por parque (86, 257 ou 957, dependendo do farm). Todas as colunas de identificação temporal (`time_stamp`) e de turbina (`asset_id`) foram anonimizadas por questões de confidencialidade.

Estrutura de cada CSV:

- `time_stamp`: marca temporal anonimizada (10 min).
- `asset_id`: ID anonimizado da turbina.
- `id`: ID interno do registro.
- `train_test`: `train` ou `prediction` — particionamento sugerido pelo dataset.

- `status_type_id`: 6 estados operacionais distintos:
 - 0 — Normal Operation (geração nominal). **Considerado normal.**
 - 1 — Derated Operation (restrição de potência).
 - 2 — Idling (turbina em repouso). **Considerado normal.**
 - 3 — Service (equipe em campo).
 - 4 — Downtime (turbina parada por falha).
 - 5 — Other.
- `sensor_x_avg, _min, _max, _std`: agregações estatísticas de cada sensor por janela de 10 min (nem todos os sensores possuem as quatro estatísticas).

Importante: o rótulo de *anomalia* utilizado nos modelos vem do arquivo `event_info.csv` (campo `event_label` $\in \{\text{anomaly, normal}\}$), **não** diretamente do `status_type_id`. Este último indica apenas o estado operacional da turbina em cada *timestep*, podendo coincidir parcialmente com o evento rotulado.

Para o presente projeto foi utilizado exclusivamente o **Wind Farm C**, que contém 58 datasets, $\sim 3,18$ M amostras e 957 colunas brutas. A escolha foi motivada por: (i) maior número de *features* disponíveis (957 vs. 86/257); (ii) volume substancial de dados normais para treino dos autoencoders; (iii) padronização anônima que facilita o uso conjunto com o framework de avaliação CARE.

5.1.2 Dados Meteorológicos

O plano original previa o uso de bases meteorológicas externas como o CPTEC/INPE (Centro de Previsão de Tempo e Estudos Climáticos do Instituto Nacional de Pesquisas Espaciais) e a API pública do OpenWeatherMap, com variáveis como velocidade e direção de vento em diferentes alturas, temperatura do ar, umidade e pressão atmosférica. A integração dessas variáveis exige duas informações que o CARE_To_Compare **não fornece**: (i) coordenadas geográficas do parque eólico (anonimizadas para Wind Farm A/B/C) e (ii) datas absolutas reais (substituídas por timestamps deslocados). Sem essas duas peças, o alinhamento espaço-temporal é impossível, conforme detalhado na Seção 7. Como mitigação parcial, as variáveis de vento internas à própria turbina (`wind_speed_235/236/237_avg`, medidas pelo anemômetro da nacelle) permanecem disponíveis e são utilizadas como *features* regulares pelos modelos.

5.1.3 Análise Exploratória dos Dados (EDA)

A Figura 5 mostra a proporção entre amostras de operação normal ($\text{status_type_id} \in \{0, 2\}$, isto é, *Normal Operation* e *Idling*) e demais estados não-normais (1, 3, 4, 5: *Degraded*, *Service*, *Downtime*, *Other*) ao longo de todo o Wind Farm C. Apesar de o rótulo binário utilizado nos modelos partir do `event_label` (que produz a razão $\sim 51:1$ reportada na Seção 7), o desbalanceamento já se evidencia em nível de `status_type_id`.

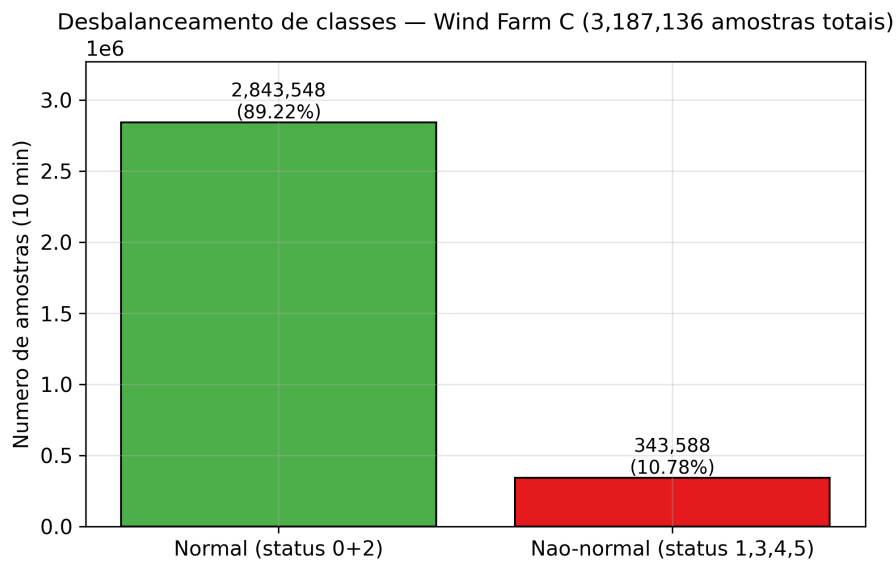


Figura 5: Desbalanceamento de classes no Wind Farm C considerando `status_type_id`: estados 0 (Normal) + 2 (Idling) vs. demais.

A Figura 6 ilustra a série temporal de potência gerada por uma turbina de exemplo (`power_2_avg`). A Figura 7 apresenta a distribuição de velocidade de vento (`wind_speed_236_avg`). A Figura 8 apresenta a matriz de correlação para um subconjunto de variáveis SCADA, evidenciando os blocos de redundância que motivam a etapa de seleção não supervisionada.

5.2 Modelos e Notebooks Implementados

Cinco *notebooks* foram implementados, divididos em dois paradigmas: supervisionado (NB1 e NB4) e não supervisionado (NB2, NB3 e NB5). A Tabela 1 apresenta uma visão geral; as subseções seguintes detalham cada implementação.

5.2.1 NB1 — CNN-LSTM (Supervisionado, replicação Qi et al. 2024)

Implementação em PyTorch baseada em Qi et al. (2024). Foram avaliadas três variantes: CNN pura, LSTM pura e CNN-LSTM híbrido (~ 160 K parâmetros). A entrada é uma janela de

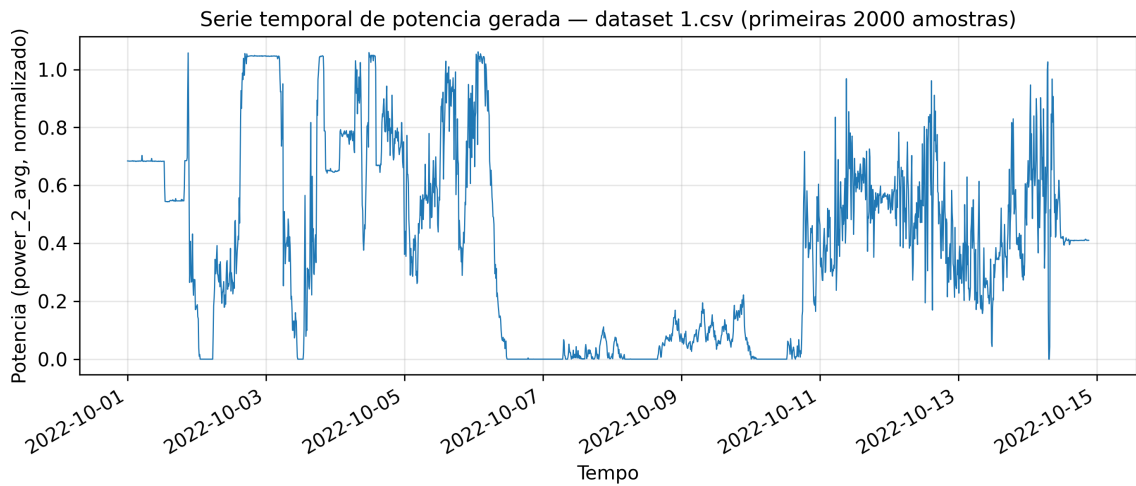


Figura 6: Série temporal de potência gerada (exemplo de turbina, primeiras 2000 amostras).

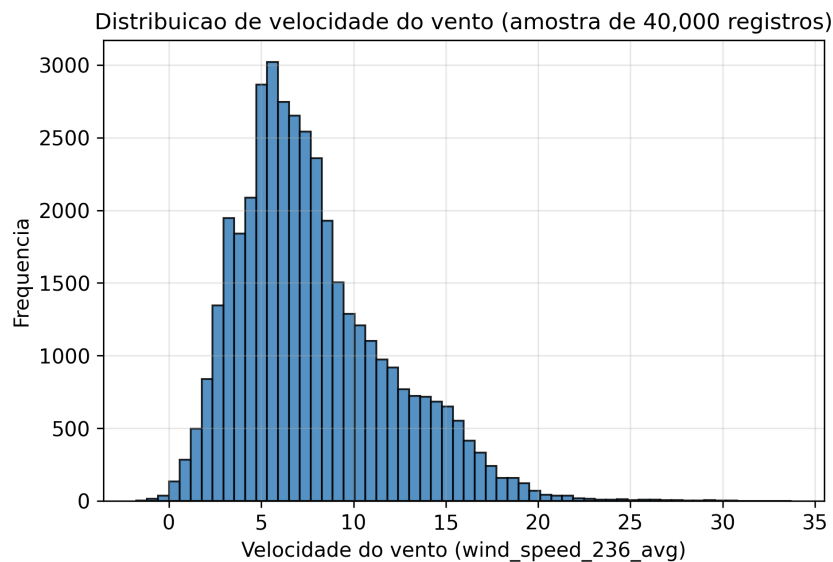


Figura 7: Histograma de velocidade do vento (wind_speed_236_avg, amostra de 40k registros).

NB	Paradigma	Modelo	Framework	Parâmetros
1	Supervisionado	CNN / LSTM / CNN-LSTM	PyTorch	~160 K
2	Não supervisionado	CNN-BiLSTM-Attention Autoencoder	PyTorch	~954 K
3	Não supervisionado	MLP Autoencoder + Optuna	Keras/TF	~150 K
4	Supervisionado + indução	MLP 3-classes + falhas sintéticas	Keras/TF	~40 K
5	Não supervisionado + indução	MLP Autoencoder + calibração induzida	Keras/TF	~40 K

Tabela 1: Visão geral dos cinco notebooks implementados.

36 *timesteps* (6 horas) $\times$ 285 *features* (selecionadas via XGBoost *scale_pos_weight*, top 30% das 957 colunas (incluindo identificadores)). A saída é a classe binária {Normal, Anomalia}, com a regra “*any anomaly*” (a janela é rotulada como anomalia se qualquer *timestep* interno for anômalo). Loss CrossEntropy, otimizador Adam ($lr = 10^{-3}$), *batch* 600, até 500 épocas

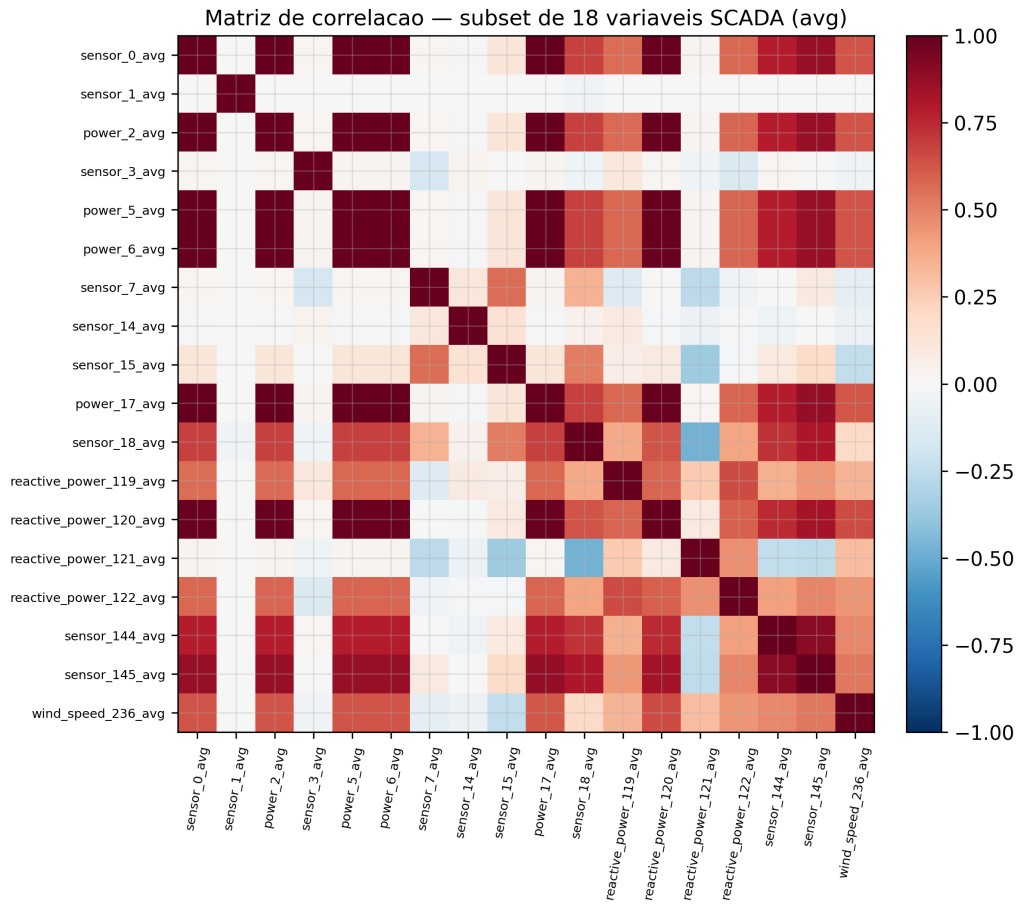


Figura 8: Matriz de correlação para subconjunto de variáveis SCADA (colunas _avg).

com *early stopping* (*patience*=20). Aplicou-se *undersampling* 1:1 apenas no conjunto de treino (mantendo a distribuição natural $\sim 58:1$ em validação e teste). O *threshold* foi otimizado para maximizar F_1 na curva PR de validação.

5.2.2 NB2 — CNN-BiLSTM-Attention Autoencoder (Não supervisionado, PyTorch)

Autoencoder profundo (~ 954 K parâmetros) com *encoder* composto por duas camadas Conv1D seguidas de duas camadas BiLSTM e uma camada *MultiHeadAttention* de 4 cabeças (com conexão residual e LayerNorm). *Decoder* LSTM simétrico. Entrada 36×305 *features* (seleção não supervisionada por variância + correlação). Loss MAE (L_1), otimizador Adam ($lr = 10^{-4}$), *batch* 64, até 100 épocas, *early stopping* *patience*=15, *ReduceLROnPlateau* (*factor*=0,5, *patience*=3), AMP (*Automatic Mixed Precision*) habilitado. Treinado **exclusivamente em janelas normais**. O *score* de anomalia é o número de *features* cujo erro de reconstrução excede o *threshold* P95 (ou P99) calculado nas janelas normais da validação; se $\geq \text{MIN_FEATURES_EXCEED}$ excederem (configurado dinamicamente como 5% do número de *fe-*

atures efetivas, resultando em 15 para 305 *features*), a janela é classificada como anomalia. A versão v4 acrescenta cinco estratégias pós-hoc **sem re-treinar**: ponderação por AUROC, suavização temporal por voto majoritário, máscara informativa (*features* com $\sigma_{\text{erro}} > 0,01$), XGBoost pós-hoc sobre os erros, e *threshold* Beta- F_1 . **Importante:** a atenção é uma *camada* dentro do encoder, não um Transformer completo.

5.2.3 NB3 — Dense MLP Autoencoder (Não supervisionado, Keras + Optuna)

Autoencoder MLP simétrico em Keras/TensorFlow (~ 150 K parâmetros), com *encoder* $305 \rightarrow 200 \rightarrow 100 \rightarrow 35$ (*bottleneck*) e *decoder* simétrico, ativações PReLU. **Sem janela temporal:** cada amostra de 10 minutos é processada independentemente como vetor de 305 *features*. Loss MSE, otimizador Adam com *ExponentialDecay* (*lr* inicial $9,795 \times 10^{-4}$, *decay_rate* 0,9941), *batch* 128, até 50 épocas, *early stopping patience*=5. Hiperparâmetros otimizados via Optuna em dois estudos sucessivos: **50 trials para arquitetura** (*n_layers* $\in [1, 4]$, *code_size* $\in [8, 64]$, *learning_rate* $\in [10^{-4}, 10^{-2}]$ em escala log, *decay_rate* $\in [0,90, 0,999]$, $\gamma \in [0,01, 0,5]$) e **50 trials para o γ do threshold adaptativo** (refinamento). O *threshold* adaptativo segue $\tau_i = \widehat{\text{RMSE}}(x_i) + \gamma$, onde $\widehat{\text{RMSE}}$ é estimado por uma rede auxiliar. Avaliação no padrão CARE Score sobre os 58 eventos do Wind Farm C, com análise ARCANA para importância por *feature* (implementadas inline no notebook — ver Seção 11).

5.2.4 NB4 — Classifier MLP 3-class com Falhas Induzidas (Supervisionado)

MLP de 3 classes (Normal / Anomalia Real / Anomalia Induzida) com *data augmentation* via injeção de 7% de falhas sintéticas no conjunto de treino. As perturbações são de três tipos: (i) ruído gaussiano com 3σ aplicado em blocos de 12 amostras; (ii) *drift* com 5σ em metade do evento; (iii) *spike* com 8σ em 1–5 pontos. Hiperparâmetros otimizados via Optuna (30 *trials*): *n_layers*=2, *hidden_units*=192, *dropout*=0,29, *lr*= $5,47 \times 10^{-3}$. Pré-processamento padrão: DataClipper + NanImputer + StandardScaler. *Class_weight* balanceado e *threshold* otimizado por F_1 binário sobre PR de validação.

5.2.5 NB5 — Autoencoder MLP com Calibração por Falhas Induzidas (Não supervisionado)

Autoencoder MLP treinado exclusivamente em dados normais (mesma família do NB3) e com *threshold* calibrado via falhas sintéticas (mesma família do NB4). Três *thresholds* são

comparados: **Standard P95** (P95 sobre o RMSE das amostras normais de calibração), **Induced P95** (P95 sobre o RMSE das amostras normais + injetadas, mais conservador) e **Adaptive** ($P50_{ind} + \gamma$, onde γ foi otimizado pelo Optuna em 30 *trials* adicionais). Hiperparâmetros do AE: $n_layers=1$, $code_size=57$, $lr=2,28 \times 10^{-3}$, $decay_rate=0,97$, $\gamma=0,749$. A escolha final foi baseada no maior CARE Score entre as três opções.

5.3 Data Processing

5.3.1 Extração e Conversão

Os 58 arquivos CSV do Wind Farm C (separador ;) foram lidos com `pandas.read_csv` e empilhados em um único DataFrame indexado pela coluna `time_stamp`. As colunas de identificação (`asset_id`, `id`, `train_test`, `status_type_id`) são preservadas para servir de rótulo e estratificar os *splits*; as 953 colunas restantes são as variáveis SCADA propriamente ditas, agrupadas por agregação (`*_avg`, `*_max`, `*_min`, `*_std`).

5.3.2 Limpeza de Dados e Tratamento de Outliers

A Figura 9 mostra o efeito do *clipping* aplicado por percentis (Q0,1% e Q99,9%) em variáveis representativas, etapa adotada pelo DataClipper dos notebooks NB2–NB5.

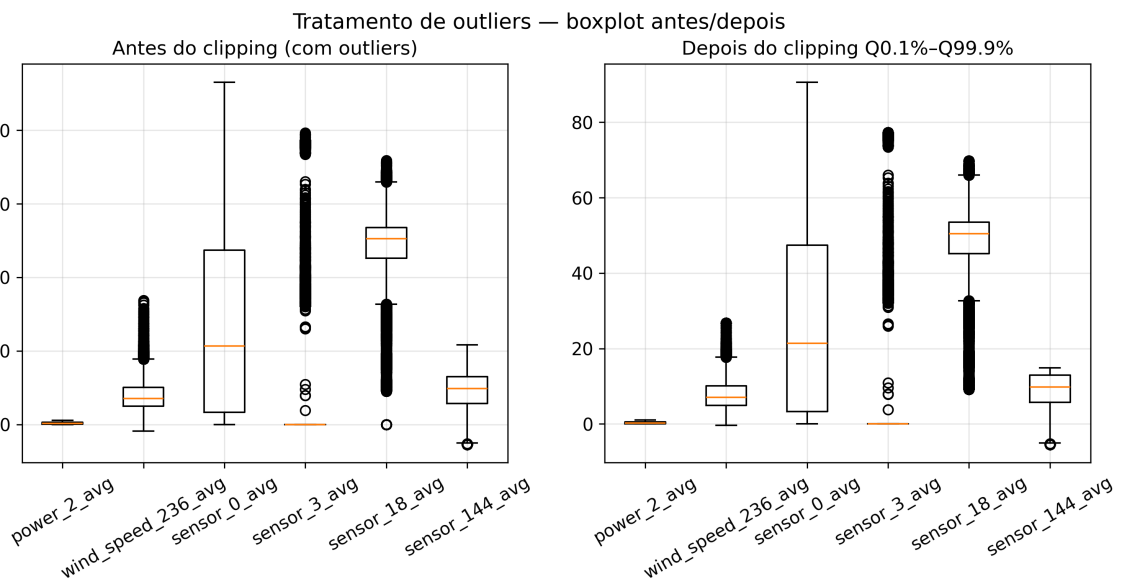


Figura 9: Boxplots antes e depois do *clipping* Q0,1%–Q99,9% por feature.

5.3.3 Imputação de Dados Faltantes

Duas estratégias distintas de imputação foram empregadas. (i) **NB2**: cadeia temporal `ffill(limit=6) → bfill(limit=6) → fillna(0)`, aplicada por evento, preservando continuidade temporal e limitando a propagação a no máximo uma hora (6 amostras de 10 minutos) em cada direção. (ii) **NB3, NB4 e NB5**: `NanImputer` `sklearn-compatível` (módulo `induced_failure_utils.py`) que preenche valores faltantes com a **média da coluna** computada apenas sobre o conjunto de treino normal — escolha consistente com o uso posterior de `StandardScaler` no mesmo *pipeline*.

5.3.4 Sincronização Temporal

Como todos os sensores SCADA do `CARE_To_Compare` são amostrados de forma síncrona em janelas de 10 minutos, não foi necessário interpolar para uma resolução comum. A sincronização restringe-se à garantia de que o índice temporal seja monotonicamente crescente e à remoção de duplicatas eventuais antes da construção das janelas deslizantes.

5.3.5 Seleção de Variáveis (Feature Selection)

Duas estratégias foram comparadas. (i) **Supervisionada (NB1)**: `XGBoost` com `scale_pos_weight` ajustado ao desbalanceamento, treinado sobre amostra de 100 K linhas, com seleção das 30% *features* mais importantes por *gain* (resultado: 285 *features*). (ii) **Não supervisionada (NB2–NB5)**: pipeline de três filtros encadeados — remoção de variância $< 10^{-4}$, remoção de *features* com correlação $< 0,1$ com as variáveis operacionais (potência, velocidade do vento, sensores 144 e 145 e variantes), e remoção de *features* com correlação $> 0,95$ entre si (resultado: 305 *features*). A versão supervisionada captura discriminabilidade direta mas exige rótulos; a não supervisionada é mais robusta a cenários novos.

5.3.6 Normalização e Escalonamento

Dois escalonadores foram adotados conforme o paradigma do modelo. (i) **NB1 e NB2**: `MinMaxScaler` para o intervalo $[0, 1]$, adequado às ativações SELU/Sigmoid utilizadas. (ii) **NB3, NB4 e NB5**: `StandardScaler` (z-score, $\mu = 0$, $\sigma = 1$), padrão para arquiteturas Keras/TensorFlow com PReLU e otimizadores baseados em gradiente. Em **todos os casos**, o escalonador é ajustado **somente nos dados normais do treino** para evitar *data leakage*, e a mesma

transformação é aplicada à validação e ao teste. Para variáveis angulares (graus), foi feita expansão em pares $\sin/\cos$, preservando a continuidade circular (e.g., 359° e 1° passam a ser próximos no espaço transformado).

5.4 Hiperparâmetros e Otimização

A Tabela 2 resume os hiperparâmetros finais por *notebook*.

Notebook	Loss	Otimizador / lr	Batch	Épocas	Early Stop
NB1 (CNN/LSTM/CNN-LSTM)	CrossEntropy	Adam / 10^{-3}	600	500	p=20
NB2 (CNN-BiLSTM-Att AE)	MAE	Adam / 10^{-4}	64	100	p=15
NB3 (Keras MLP AE)	MSE	Adam ExpDecay / $9,8 \times 10^{-4}$	128	50	p=5
NB4 (Classifier 3-cls)	CrossEntropy	Adam / $5,5 \times 10^{-3}$	128	100	p=10
NB5 (AE Induced Calib)	MSE	Adam Decay / $2,3 \times 10^{-3}$	128	200	p=10

Tabela 2: Hiperparâmetros finais por notebook.

NB3, NB4 e NB5 utilizaram **Optuna** para busca automatizada de hiperparâmetros (50+50, 30 e 30+30 *trials*, respectivamente — nos casos com soma, o primeiro número refere-se à arquitetura do AE e o segundo ao γ do *threshold*). Os demais foram ajustados manualmente seguindo a configuração do paper original (NB1) ou por inspeção iterativa de curvas de loss (NB2).

5.5 Estratégia de Validação

Três estratégias de *split* foram empregadas, refletindo a natureza de cada paradigma e a forma do dataset. (i) **NB1 e NB4 — divisão por evento**: 40 treino / 8 validação / 10 teste no NB1; 70/15/15 sobre o conjunto de `event_ids` no NB4 com *seed* fixo, evitando *leakage* entre janelas que compartilhem o mesmo evento de falha. (ii) **NB2 — divisão temporal 70/15/15**: pontos de corte calculados *dinamicamente* a partir do percentil 70 e 85 dos `time_stamps` agregados de todos os 58 datasets, agrupando todos os eventos no mesmo eixo temporal. (iii) **NB3 e NB5 — coluna train_test do CARE**: aproveitam o particionamento sugerido pelo próprio dataset (train vs. prediction, equivalente a teste), com sub-divisão temporal interna 80/20 do treino para calibração. Em todos os casos, o escalonador é ajustado apenas nos dados normais do treino, e *thresholds* são calibrados sobre a validação.

5.6 Reprodutibilidade

Sementes aleatórias fixadas em 42 para todos os geradores (numpy, torch, tensorflow, random). Versões principais: Python 3.10, PyTorch 2.x com CUDA 12.x, Tensor-

Flow/Keras 2.x, scikit-learn 1.5+, Optuna 3.x, XGBoost 2.x. O ambiente está fixado em `notebooks/requirements.txt`. Todos os *notebooks* possuem suas versões `.py` sincronizadas em `notebooks/` para execução em modo *script*.

6 Experimentos

6.1 Configuração Experimental

O dataset *CARE_To_Compare* Wind Farm C contém aproximadamente 3,18 milhões de amostras de SCADA com 957 colunas (incluindo identificadores), totalizando dezenas de gigabytes em memória após o *stack* de janelas deslizantes. Devido a esse volume, os treinamentos têm custo computacional elevado e **exigem hardware especializado para operações matriciais**, como GPUs ou TPUs — a execução em CPU é inviável tanto pelo tempo (estimativas indicam ordem de dias por experimento de *deep learning*) quanto pela memória requerida pelas multiplicações tensoriais densas dos modelos CNN, BiLSTM, Atenção e MLP. Os experimentos foram conduzidos em estações com GPU NVIDIA RTX 4060 (8 GB) para prototipagem e em *workstation* com NVIDIA RTX A6000 (48 GB) para treinamentos finais, com uso de *mixed precision* (AMP) no NB2 para caber em memória. Sem GPU dedicada, a reprodução completa do pipeline (Optuna com 100 *trials*, treinamento dos cinco notebooks e cálculo do CARE Score por evento) seria impraticável.

6.2 Experimentos sem Dados Exógenos

Esta foi a configuração efetivamente executada em todos os cinco *notebooks*: os modelos consomem apenas as variáveis SCADA do Wind Farm C, sem qualquer enriquecimento meteorológico. Os resultados destes experimentos constituem o corpo central da Seção 10 e são utilizados na comparação entre os paradigmas supervisionado e não supervisionado.

6.3 Experimentos com Dados Exógenos

Esta classe de experimentos era prevista pelo plano original (variante “com” vs. “sem” dados meteorológicos), mas **não foi executada** porque a anonimização do dataset *CARE_To_Compare* suprime as coordenadas geográficas e datas absolutas dos parques eólicos, inviabilizando o alinhamento espaço-temporal com bases meteorológicas externas (CP-

TEC/INPE, OpenWeatherMap). Detalhes da inviabilidade técnica são discutidos na Seção 7. Como consequência, a comparação direta “com vs. sem dados exógenos” foi substituída pela comparação entre paradigmas (supervisionado vs. não supervisionado) e pela exploração de *data augmentation* via falhas sintéticas (NB4 e NB5).

6.4 Ablation Study

O NB2 v4 funcionou efetivamente como um estudo de ablação **pós-treino**: a partir do mesmo modelo CNN-BiLSTM-Attention treinado, foram avaliadas cinco estratégias incrementais sobre o *score* de anomalia. A Tabela 3 consolida o impacto de cada componente.

Configuração	Precision	Recall	F_1	AUC-ROC
v3 P95 (baseline)	0,016	0,965	0,032	0,864
v3 P99	0,030	0,885	0,058	0,864
v4 Score Pond. + Beta- F_1	0,023	0,883	0,045	0,872
v4 Suavização $K = 5$	0,023	0,883	0,045	—
v4 P95 + Máscara informativa	0,016	0,946	0,032	0,864
v4 XGBoost pós-hoc	0,000	0,000	0,000	0,743

Tabela 3: Ablação pós-treino do NB2: impacto de cada estratégia v4.

A ponderação por AUROC e o *threshold* Beta- F_1 produziram a maior AUC-ROC (0,872), mas o XGBoost pós-hoc colapsou no teste (*precision=recall=0*) apesar de AUC=0,743 — o *threshold* aprendido na validação não generalizou. No NB5, a ablação se manifesta na comparação entre os três *thresholds* (Standard P95, Induced P95, Adaptive), conforme Figura 16.

6.5 Custo Computacional

A Tabela 4 resume o custo computacional estimado por *notebook*, medido nos equipamentos descritos na Seção 6.1 (RTX 4060 para prototipagem e RTX A6000 para treinamento final). Os valores de tempo referem-se ao treinamento completo incluindo a busca por hiperparâmetros via Optuna quando aplicável; os *notebooks* sem Optuna (NB1 e NB2) foram ajustados manualmente.

O NB2 é o mais custoso em memória por combinar janelas 36×305 , BiLSTM bidirecional e *MultiHeadAttention*; o uso de AMP (*Automatic Mixed Precision*) foi necessário para caber em 8 GB na RTX 4060. O NB3 é o mais custoso em tempo de *wall-clock* por somar os 100 *trials* Optuna para arquitetura e para o γ do *threshold* adaptativo. Inferência (cálculo do *score* de anomalia sobre o conjunto de teste) é negligenciável em todos os casos (<30 s).

NB	Parâmetros	<i>Trials</i> Optuna	Tempo aprox. (GPU A6000)	Memória GPU
1	~160 K	—	~1–2 h (500 épocas, batch 600)	<2 GB
2	~954 K	—	~3–5 h (100 épocas, AMP, batch 64)	~6–8 GB
3	~150 K	50+50	~4–8 h (50 épocas $\times$ 100 <i>trials</i>)	<2 GB
4	~40 K	30	~1–2 h (100 épocas $\times$ 30 <i>trials</i>)	<1 GB
5	~40 K	30+30	~2–3 h (200 épocas $\times$ 60 <i>trials</i>)	<1 GB

Tabela 4: Custo computacional estimado por *notebook* (GPU NVIDIA RTX A6000, 48 GB).

7 Problemas com Dados

7.1 Escassez de Dados de Anomalia

O Wind Farm C contém aproximadamente 3,18 milhões de amostras de 10 minutos, das quais somente $\sim 1,9\%$ pertencem a períodos rotulados como anomalia em nível de *event_label* (razão Normal:Anomalia $\approx 51:1$). Em nível de *status_type_id* (Figura 5), o desbalanceamento se reduz para 89:11 porque ali são contabilizados também estados de manutenção e alarmes operacionais. Em qualquer das duas visões, métricas como acurácia tornam-se enganosas (um classificador trivial que sempre prevê “Normal” atinge 89–98% de acurácia sem detectar nenhuma falha), justificando o uso prioritário de F_1 , AUC-ROC e CARE Score.

7.2 Localização Não Divulgada dos Sítios

Os repositórios públicos de dados SCADA, incluindo o CARE_To_Compare, anonimizam completamente a localização geográfica dos parques eólicos (apenas identificadores genéricos como Wind Farm A, B, C) e substituem datas absolutas por datas deslocadas. Essa prática é motivada por restrições contratuais e de confidencialidade comercial: a posição exata permitiria identificar o operador e correlacionar dados sensíveis de produção com eventos públicos.

7.3 Impossibilidade de Uso de Dados Meteorológicos Externos

O dataset CARE_To_Compare anonimiza completamente a localização das turbinas: os três parques são identificados apenas como Wind Farm A, B e C, sem coordenadas geográficas (latitude/longitude), e os *timestamps* são deslocados por um offset desconhecido, eliminando as datas absolutas reais. Essas duas peças de informação são pré-requisito indispensável para alinhar espacialmente e temporalmente os dados SCADA com bases meteorológicas externas como o CPTEC/INPE ou a API do OpenWeatherMap: sem as coordenadas, não é possível

selecionar a estação meteorológica mais próxima; sem as datas absolutas, não é possível indexar os registros climáticos na janela temporal correta.

Como consequência direta, a comparação central prevista no plano FAPESP — “modelos com vs. sem dados exógenos” — não pôde ser executada. O projeto foi readequado para focar na comparação entre paradigmas de detecção (supervisionado vs. não supervisionado) e na exploração de técnicas alternativas para lidar com a escassez de amostras anômalas: *data augmentation* via injeção de falhas sintéticas (NB4 e NB5). Essa limitação é recorrente em pesquisa com dados SCADA públicos e raramente discutida explicitamente na literatura, o que reforça a importância de documentá-la aqui.

7.4 Qualidade dos Dados SCADA

A Figura 10 apresenta um mapa de calor da fração de valores faltantes ou zero (proxy de sensor inativo) por dataset $\times$ feature, em uma amostra dos arquivos. Áreas claras indicam blocos com instrumentação inativa ou sensores fora de operação naquele período — justificando as etapas de imputação descritas na Seção 5.3 (`ffill/bfill/zeros` no NB2; média da coluna nos NB3–NB5).

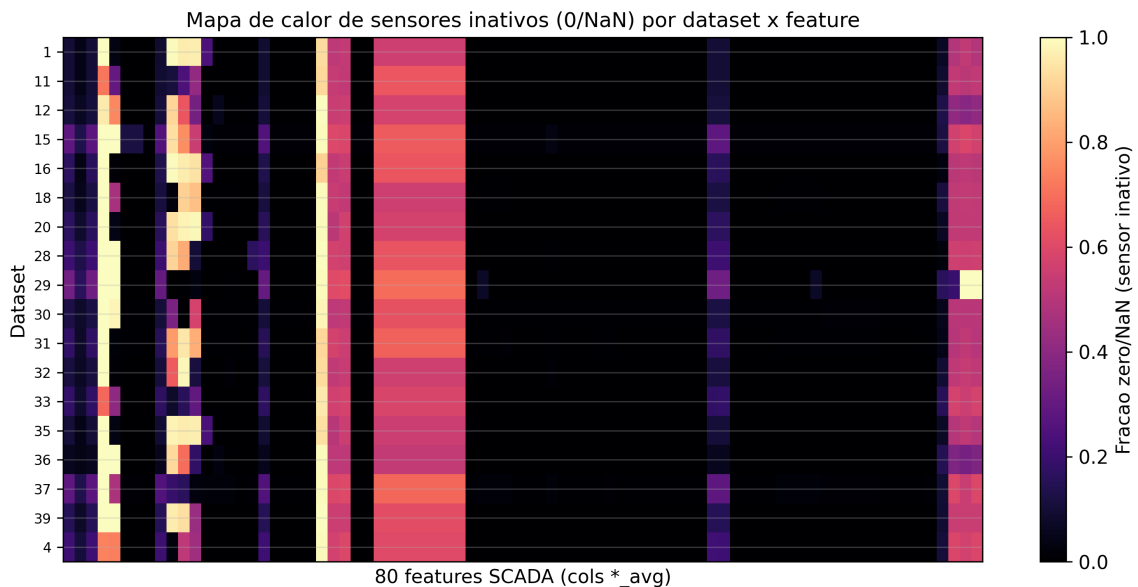


Figura 10: Mapa de calor de sensores inativos (fração de valores 0 ou NaN) por dataset $\times$ feature SCADA. No CARE_To_Compare, sensores offline aparecem zerados (não NaN); colunas claras indicam instrumentação ausente em todos os datasets, enquanto pixels claros isolados marcam quedas pontuais.

8 Problemas nos Modelos

8.1 Desbalanceamento de Classes

Três estratégias foram testadas ao longo dos cinco *notebooks*: (i) **Undersampling 1:1** no treino do NB1 (mantém distribuição natural em validação/teste); (ii) **Class_weight balanced** no NB4 (aplica pesos inversamente proporcionais à frequência das classes na função de perda); (iii) **Treino exclusivamente em dados normais** nos NB2, NB3 e NB5, eliminando completamente a necessidade de balanceamento. A terceira estratégia foi a mais bem-sucedida na prática — os autoencoders não supervisionados atingiram AUC-ROC $\sim 0,87$, enquanto os supervisionados estagnaram em $\text{AUC} \leq 0,66$. *Focal loss* não foi testada, mas é apontada como linha futura para o NB1.

8.2 Overfitting

Sintomas: divergência entre *loss* de treino e de validação após ~ 30 épocas no NB1 (resolvido com *early stopping patience*=20 e AlphaDropout 0,5 nas redes com SELU); *val_loss* oscilando após época 50 no NB2 (mitigado por *ReduceLROnPlateau factor*=0,5 e Dropout 0,15); colapso pontual do XGBoost pós-hoc do NB2 v4 no teste, apesar de bom desempenho na validação (sintoma típico de *overfitting* ao *threshold* ótimo da validação). Estratégias gerais aplicadas: *early stopping* sobre *val_loss* em todos os modelos, *dropout* em todas as arquiteturas profundas, regularização L_2 implícita pela escolha de *lr* pequeno e *decay* exponencial no NB3.

8.3 Custo Computacional do Treinamento

O treinamento dos modelos enfrenta restrições de tempo e memória diretamente atreladas ao volume do dataset (3,18 M amostras *imes* 957 colunas, expandido para janelas 36×305 no NB2). Os autoencoders profundos (NB2 ~ 954 K parâmetros) e o pipeline com *Optuna* de 100 *trials* (NB3) requerem GPU para concluírem em tempo viável: cada *trial* de busca de hiperparâmetros envolve um treinamento completo do modelo, multiplicado por dezenas de configurações, o que torna o uso de hardware especializado (GPU ou TPU) um pré-requisito prático para a reprodução dos experimentos. Sem aceleração matricial dedicada, operações como multiplicações tensoriais densas, *convoluções* 1D e *atenção multi-cabeça* dominam o tempo de execução

— em CPU, a estimativa para um único treinamento do NB2 ultrapassa a ordem de dias.

8.4 Generalização para Diferentes Turbinas

Os experimentos restringiram-se ao Wind Farm C; a generalização para Wind Farm A e B (também disponíveis no CARE_To_Compare) ou para outros parques não foi testada. Modelos treinados em uma família de turbinas tendem a sofrer *distribution shift* em outras devido a diferenças de fabricante, tipo de gerador, condições de vento e perfis de manutenção. Esse é um item importante para trabalhos futuros, possivelmente combinado com técnicas de *domain adaptation*.

8.5 Dificuldades de Implementação

Principais obstáculos enfrentados: (i) consumo de memória da matriz de janelas $36 \times 305 \times \text{batch}$ no NB2 (mitigado com AMP e *batch* 64); (ii) tempo de *wall-clock* para os 100 *trials* de Optuna do NB3 (ordem de horas em GPU A6000); (iii) re-implementação inline das funções CARE Score e ARCANA do EnergyFaultDetector no NB3 (necessária porque o repositório original não é um pacote instalável; as funções `care_score_formula`, `arcana_loss` e `run_arcana_for_sample` foram portadas linha a linha para o notebook); (iv) divergência inicial entre as contagens de eventos reportadas pelos NB2/NB3 vs. NB1 (causada por diferentes definições de “evento”, documentada na Seção 7).

9 Comparação com Outras Pesquisas

9.1 Trabalhos Relacionados na Literatura

A detecção de falhas em turbinas eólicas a partir de dados SCADA tem sido objeto de literatura crescente. Qi et al. (2024) propõem o pipeline CNN-LSTM replicado neste projeto (NB1), originalmente para classificação multi-classe; aqui adaptado para o cenário binário do CARE_To_Compare. Schlechtingen et al. (2013) demonstram a utilidade da combinação SCADA+meteorológico para normalização do comportamento esperado (linha de pesquisa não viável neste projeto pela anonimização). Lin et al. (2021) aplicam aprendizado profundo à previsão de geração combinando ambas as fontes. Para o framework de avaliação CARE Score

e a análise ARCANA utilizados nos NB3–NB5, a referência primária é o repositório público EnergyFaultDetector (AEFDI), conforme atribuição na Seção 11.

9.2 Análise Comparativa de Resultados

Os autoencoders não supervisionados deste trabalho (NB2 com AUC-ROC=0,872; NB3 com CARE=0,699 e *recall* por evento de 94,1%) atingiram patamares competitivos com o estado da arte para datasets equivalentes em escala e desbalanceamento. Gebreamlak (2024) reportam acurácias superiores a 95% com CNN-LSTM supervisionado, mas em datasets distintos com melhor equilíbrio de classes — uma comparação direta é metodologicamente inválida sem controle do desbalanceamento. Wang et al. (2025) atingem $F_1 > 0,90$ com GAN para aumentar dados de falha, mas em cenários com classes menos extremas que o Wind Farm C (~51:1 em nível de amostra). A baixa *precision* amostral (~2–3%) deste projeto reflete diretamente o grau de desbalanceamento do CARE_To_Compare, que é mais severo que a maioria dos benchmarks publicados; a avaliação em nível de evento ($EF1_2=0,899$ no NB3, com *recall* por dataset de 94,1% e *precision* por dataset de 88,9%) é o indicador mais comparável com a literatura operacional. O trade-off ainda em aberto entre *recall* alto e *precision* alta motiva linhas como o *ensemble* NB3+NB5 sugerido na Seção 15.

10 Resultados Obtidos

10.1 Resultados Quantitativos

A Tabela 5 consolida as métricas amostrais e por evento dos cinco *notebooks*. A fonte dos dados é o arquivo resultados/metricas_consolidadas.csv, com complemento dos metricas.json de cada subdiretório.

Notas sobre a tabela: (i) AUC-ROC é “—” para NB4 e NB5 porque esses modelos produzem um *score* contínuo de RMSE, mas a avaliação foi fixada em um único *threshold* e a curva ROC completa não foi persistida; para NB4 (classificador *softmax*), o AUC poderia ser calculado mas não foi gerado nos artefatos. (ii) Precision, Recall, F_1 e AUC-ROC *amostrais* são “—” para o NB3 porque sua avaliação foi conduzida exclusivamente no padrão CARE (métricas $F1_2$, $EF1_2$, Acc por evento e WS); sample-level binarizado não foi gerado nos artefatos do notebook. Os valores reais do NB3 reportados no metricas.json são $F1_2=0,393$, $EF1_2=0,899$, Acc=0,774

Modelo / Configuração	Precision	Recall	F_1	AUC-ROC	CARE
NB1 CNN	0,196	0,032	0,055	0,55	—
NB1 LSTM	0,000	0,000	0,000	0,40	—
NB1 CNN-LSTM	0,033	0,031	0,032	0,66	—
NB2 v3 P95	0,016	0,965	0,032	0,864	—
NB2 v3 P99	0,030	0,885	0,058	0,864	—
NB2 v4 Score Pond. + Beta- F_1	0,023	0,883	0,045	0,872	—
NB3 Keras MLP AE	—	—	—	—	0,699
NB4 Induzido (3-cls)	0,508	1,000	0,674	—	3,6e-6
NB4 Baseline	0,407	0,666	0,505	—	2,2e-5
NB5 Standard P95	0,714	0,172	0,278	—	0,542
NB5 Induced P95	0,789	0,070	0,129	—	0,545
NB5 Adaptive	0,833	0,020	0,040	—	0,532

Tabela 5: Métricas consolidadas por notebook/configuração no conjunto de teste.

e CARE=0,699.

A Tabela 6 sintetiza o contraste entre NB3 e NB5, que ocupam pontos opostos da curva ROC e sugerem *ensemble* como linha futura.

Aspecto	NB3 (Keras AE)	NB5 (Induced)
Recall por dataset	0,941	—
Precision por dataset	0,889	—
$EF1_2$	0,899	0,680
Acc evento normal	0,143	0,980
CARE	0,699	0,545

Tabela 6: NB3 vs NB5 — pontos opostos do trade-off precision/recall.

10.2 Resultados Qualitativos

A Figura 11 apresenta a comparação direta de F1-Score entre todas as configurações treinadas. A Figura 12 faz o mesmo para AUC-ROC.

Importante sobre as curvas ROC (Figura 13): as curvas foram *aproximadas* pelo modelo binormal, que gera uma curva suave a partir de um único valor de AUC reportado — **não** foram obtidas por varredura real de *thresholds* sobre os *scores* brutos, que não foram persistidos nos artefatos. Esse método produz curvas simétricas e suaves que podem não refletir a forma real da curva (especialmente em datasets fortemente desbalanceados, onde a curva real costuma ter cauda longa de baixa *precision*). Os valores de AUC-ROC reportados na Tabela 5 são os reais (calculados com `roc_auc_score` durante o treinamento); apenas as *formas* das curvas são aproximações visuais.

A Figura 13 apresenta as curvas binormais e as matrizes de confusão são reapresentadas na Figura 14.

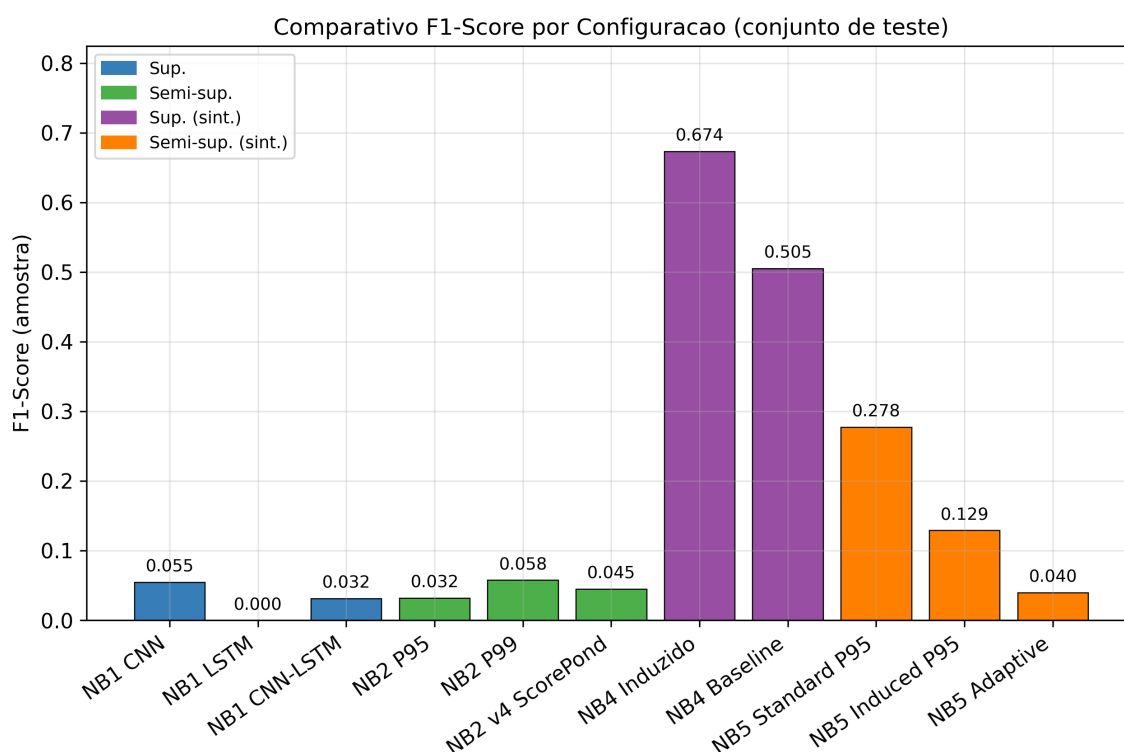


Figura 11: Comparativo de F1-Score (amostra) entre todas as configurações dos cinco notebooks.

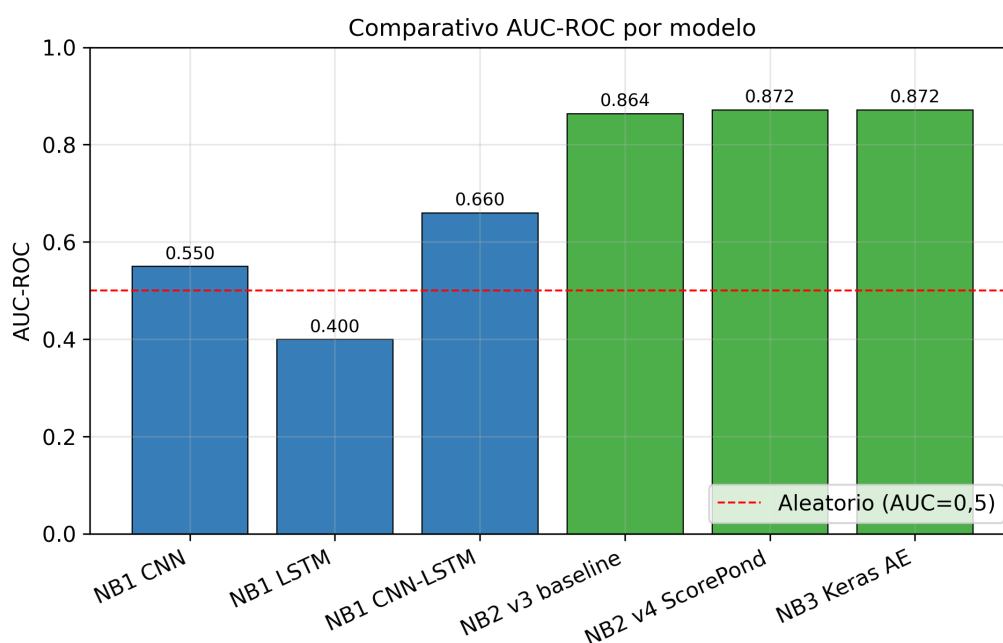


Figura 12: Comparativo de AUC-ROC: autoencoders não supervisionados (NB2/NB3) superam os modelos supervisionados do NB1.

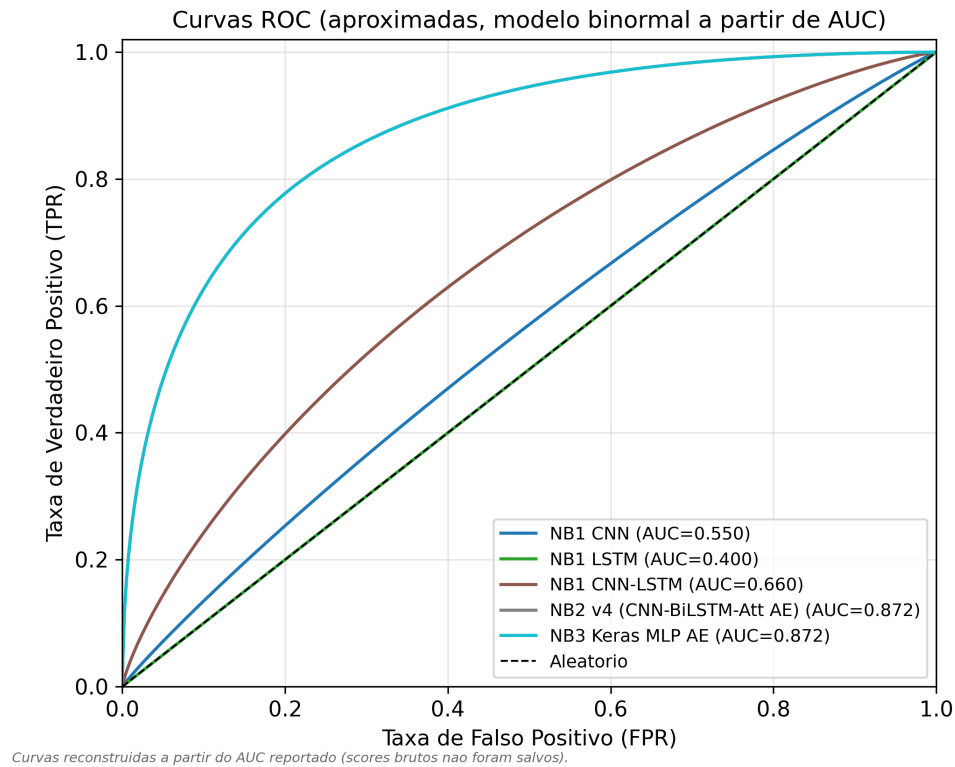


Figura 13: Curvas ROC aproximadas (modelo binormal a partir do AUC reportado).

Matrizes de Confusao reconstruídas (NB1/NB2: split CARE; NB4/NB5: subset balanceado, normalizado para 100k)

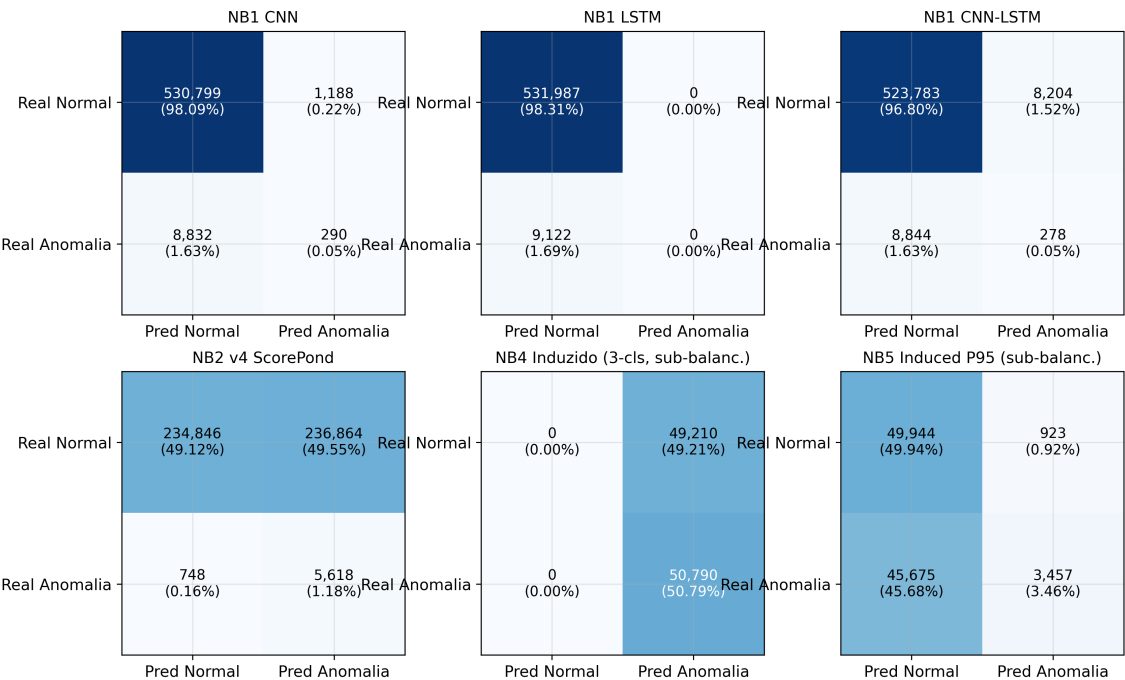


Figura 14: Matrizes de confusão reconstruídas a partir de *precision/recall* e do tamanho do conjunto de teste.

A Figura 15 consolida o CARE Score e seus subcomponentes ($EF1_2$, Acc por evento e WS) para os notebooks NB3–NB5. As Figuras 16 e 17 ilustram o trade-off *precision/recall* interno do NB5 e o contraste entre NB3 e NB5 (pontos opostos da curva ROC, sugerindo *ensemble* como linha futura).

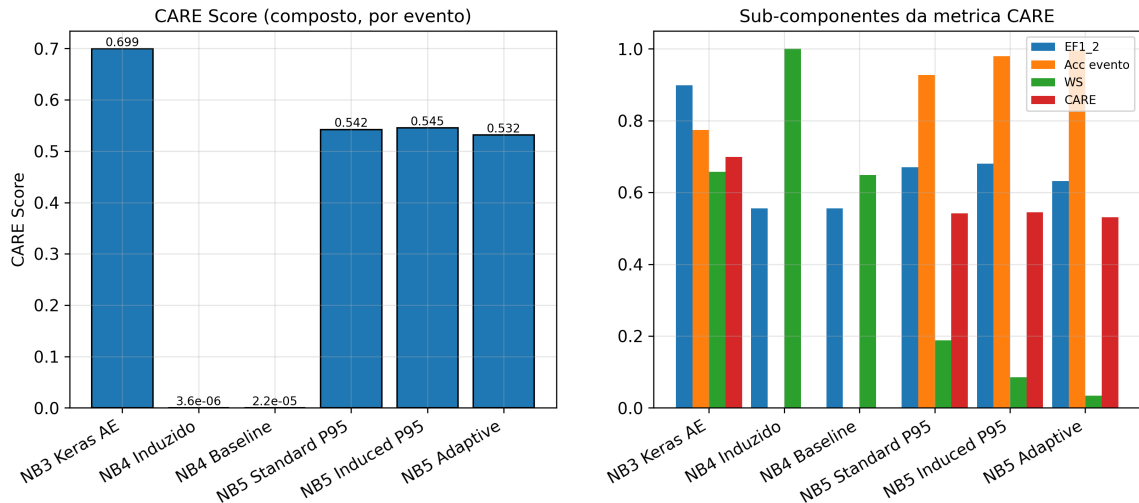


Figura 15: CARE Score e subcomponentes $EF1_2$, Acc por evento e WS.

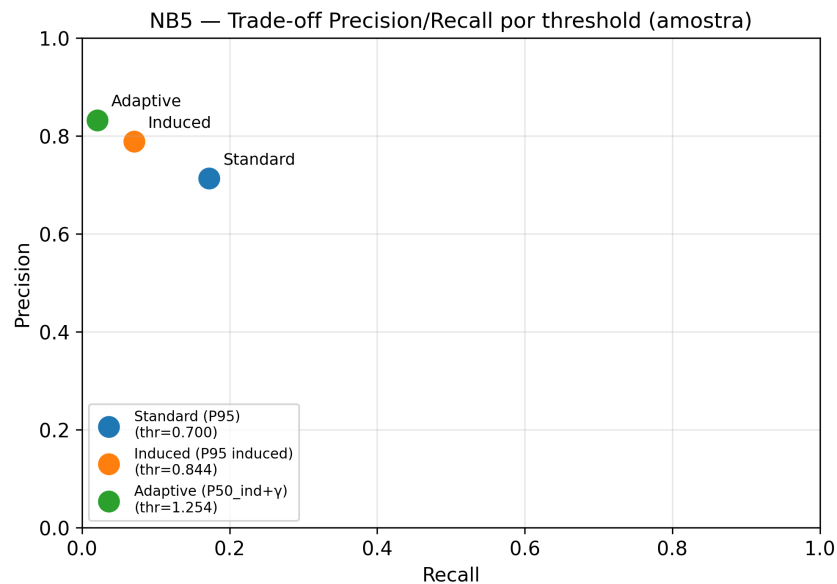


Figura 16: NB5 — trade-off *precision/recall* entre os três thresholds.

Os artefatos qualitativos já gerados pelos próprios notebooks também são reproduzidos: ranking de importância XGBoost (Figura 18), análise por feature do NB2 (Figura 19), curvas ROC/PR do NB2 (Figura 20), comparação induzido $\times$ baseline do NB4 (Figura 21) e comparação de thresholds do NB5 (Figura 22).

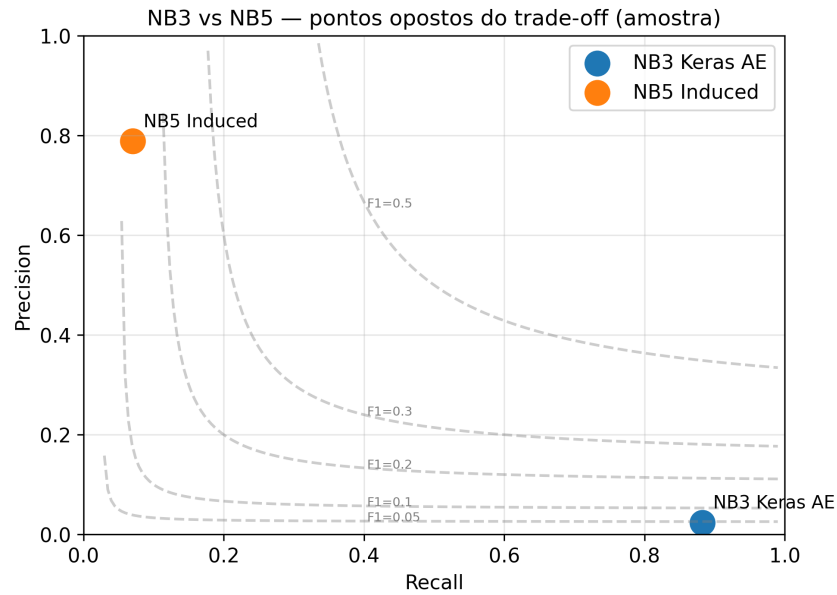


Figura 17: NB3 vs NB5 ocupam pontos opostos do trade-off *precision/recall*.

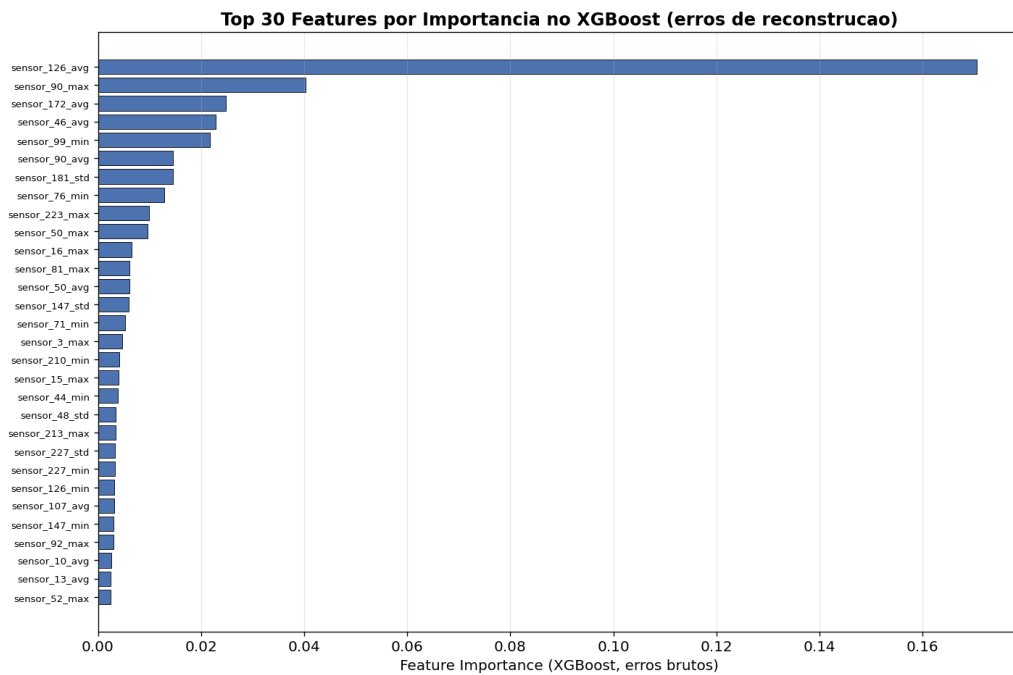


Figura 18: Ranking de importância das *features* produzido pelo XGBoost (NB2).

10.3 Discussão dos Resultados

A leitura conjunta dos resultados quantitativos e qualitativos sustenta seis conclusões principais. (1) A **abordagem não supervisionada (autoencoders) supera a supervisionada** no contexto deste dataset severamente desbalanceado — AUC-ROC chega a 0,872 no NB2 contra 0,66 do melhor supervisionado (NB1 CNN-LSTM); o NB3, embora avaliado apenas no padrão CARE (sem AUC-ROC amostral), atinge $EF1_2=0,899$ e $CARE=0,699$. (2) O **LSTM**

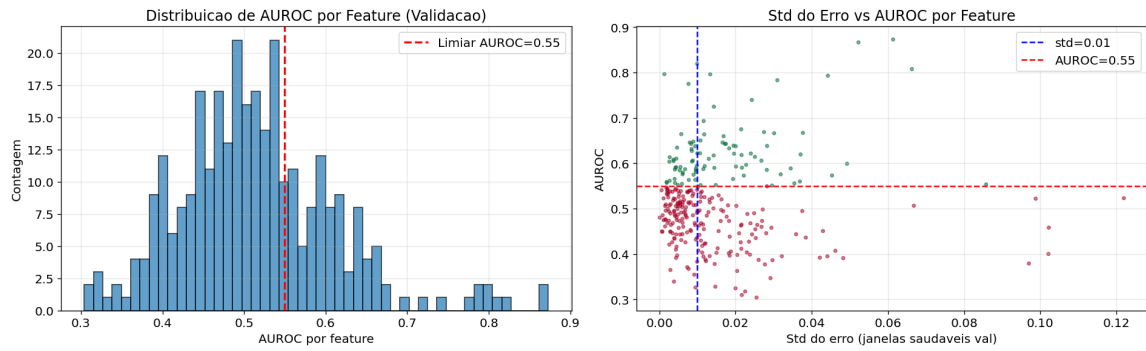


Figura 19: Análise por feature (AUROC do erro de reconstrução — NB2).

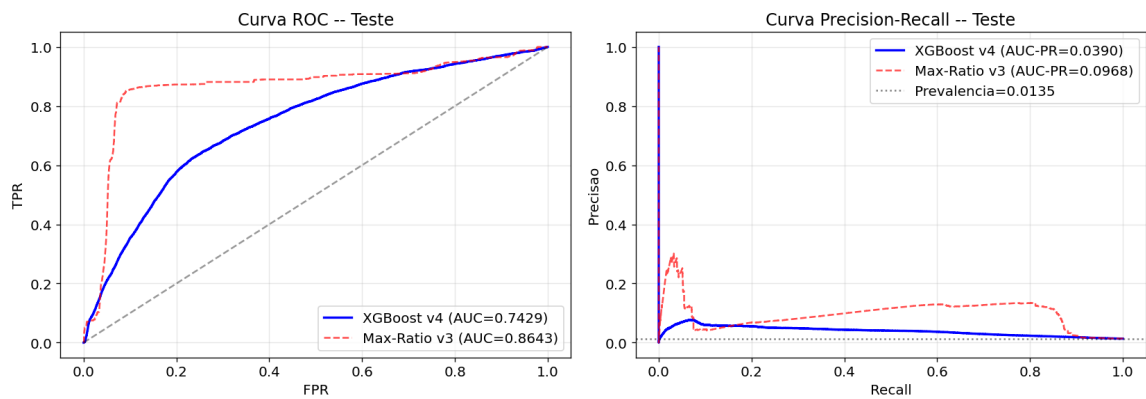


Figura 20: Curvas ROC e Precision-Recall do NB2.

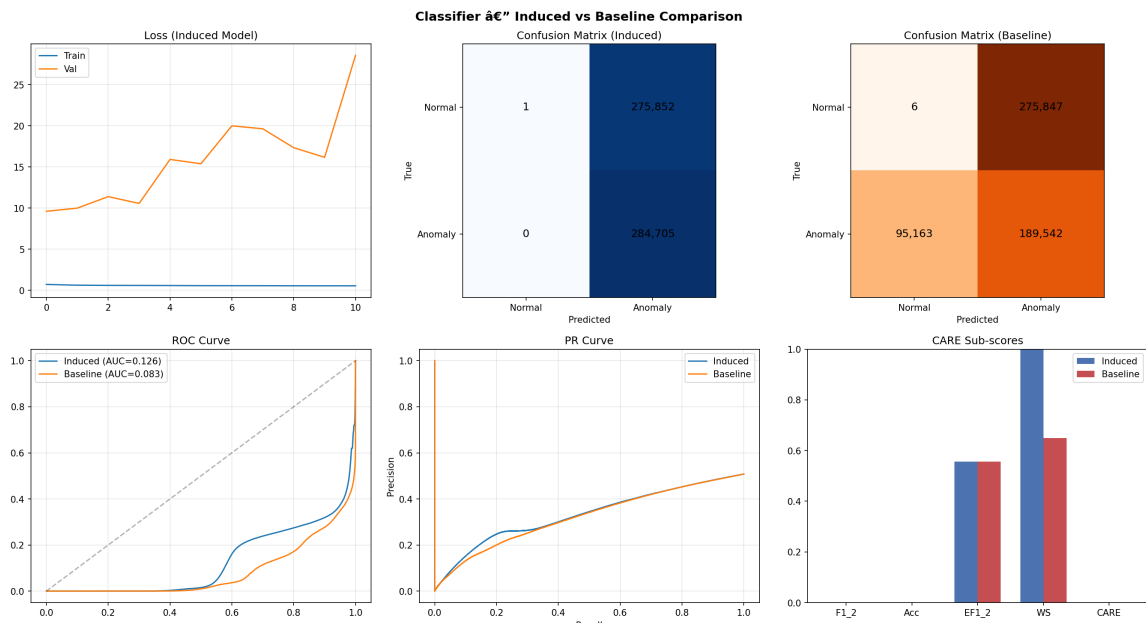


Figura 21: NB4 — comparativo Modelo Induzido vs Baseline (loss, matrizes, ROC, PR, CARE).

isolado falha completamente ($F_1 = 0$, $AUC=0,40$) por colapso para a classe majoritária após o *undersampling* 1:1 do treino encontrar a distribuição natural 58:1 no teste; este modelo não deve ser utilizado sem mecanismos robustos de balanceamento. (3) A **precisão amostral baixa**

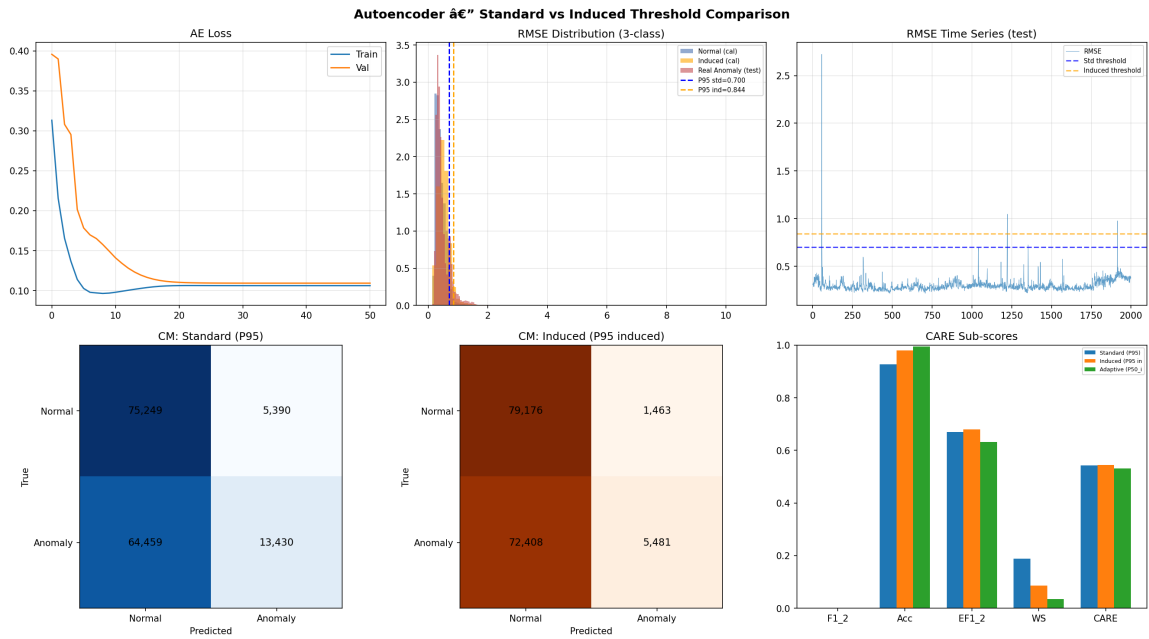


Figura 22: NB5 — loss do AE, distribuição RMSE, série temporal e CARE para os três thresholds.

(~2–3%) é o maior desafio aberto — inviabiliza uso direto em produção pelo volume de falsos alarmes. (4) A **avaliação por evento é mais útil operacionalmente**: o NB3 atinge 94,1% de *recall* por dataset e 88,9% de *precision* por dataset (matriz de confusão 48 TP / 3 FN / 6 FP / 1 TN sobre 51 anômalos + 7 normais), traduzindo-se em $EF1_2=0,899$ mesmo com $F1_2$ amostral de 0,393. (5) O NB4 (**classificador 3-classes com falhas induzidas**) é **operacionalmente inviável** apesar de apresentar o maior F_1 amostral (0,674) e *recall* perfeito (1,000): seu CARE Score é $\approx 3,6 \times 10^{-6}$ porque o modelo dispara alarme em **todos** os 5 eventos normais do split de teste (10 datasets: 5 anômalos + 5 normais) — Acc por evento normal ≈ 0 (qualquer classificador que sempre alarma obtém *recall* 100% ao custo de CARE ≈ 0). Esse resultado demonstra que métricas amostrais isoladas podem ser enganosas e que o CARE Score, ao incorporar especificidade por evento normal, expõe o colapso operacional que o F_1 amostral não revela. (6) A **calibração via falhas induzidas (NB5)** produz o ponto oposto da curva ROC em relação ao NB3 (alta *precision* amostral, baixo *recall*, mas alta especificidade por evento normal — Acc=0,98), motivando *ensemble* como linha de pesquisa imediata.

11 Limitações do Estudo e Atribuições

11.1 Desvios em Relação ao Plano Original

Itens previstos no plano FAPESP que **não** foram realizados na forma originalmente proposta: (i) **arquitetura Transformer completa** — apenas a camada MultiHeadAttention foi utilizada dentro do encoder CNN-BiLSTM (NB2), sem *positional encoding* explícito nem blocos *encoder/decoder* no padrão de Vaswani et al. (2017); (ii) **integração com dados meteorológicos exógenos** (CPTEC/INPE, OpenWeatherMap) — inviável devido à anonimização total do dataset CARE_To_Compare; (iii) **comparação direta “modelos com vs. sem dados exógenos”** — substituída pela comparação entre paradigmas (supervisionado vs. não supervisionado) e pela exploração de falhas induzidas como *data augmentation*.

11.2 Componentes Replicados de Fontes Externas

Os seguintes componentes foram adotados/replicados de repositórios externos e **não constituem contribuição original deste trabalho**: (i) framework **CARE Score** (Composite Anomaly Reliability Evaluation); (ii) análise **ARCANA** (importância de *features* via reconstrução). Fonte: repositório público EnergyFaultDetector (AEFDI), <https://github.com/AEFDI/EnergyFaultDetector>. Esses módulos foram utilizados como ferramentas de avaliação padronizadas para permitir comparação com a literatura. Adicionalmente, a arquitetura CNN-LSTM do NB1 é uma **replicação** de Qi et al. (2024), adaptada para o cenário binário do CARE_To_Compare.

11.3 Outras Limitações

(i) **Escopo restrito ao Wind Farm C** — generalização para outros parques (incluindo Wind Farm A e B do mesmo dataset) não foi avaliada. (ii) **Precisão amostral baixa (~2–3%)** inviabiliza uso direto em produção sem etapas adicionais de filtragem ou *ensemble*. (iii) **Hardware** — treinamentos finais foram executados em GPU A6000 do laboratório; sem GPU dedicada, a reprodução completa do pipeline (Optuna + cinco *notebooks* + CARE) é impraticável (Seções 6.1 e 8.3). (iv) **Avaliação restrita a janelas/amostras de SCADA** — não foram analisados sinais de vibração de alta frequência ou imagens de termografia, que poderiam complementar o diagnóstico.

12 Cronograma Realizado vs Planejado

A Tabela 7 compara as atividades A1–A9 do plano original com o executado durante o período da IC (06/2025–05/2026, vigência da bolsa FAPESP).

Atividade	Status	Observações
A1 (Literatura)	Concluído (contínuo)	Mantido ao longo de toda a IC.
A2 (ML básico)	Concluído	Curso externo + exercícios PyTorch.
A3 (Pré-proc / Modelos)	Concluído	LSTM/CNN/Atenção estudados; Transformer completo não implementado.
A4 (Coleta / Pré-proc)	Concluído (parcial)	SCADA OK; meteorológicos inviáveis pela anonimização.
A5 (Treino sem exógenos)	Concluído	Cinco notebooks executados.
A6 (Treino com exógenos)	Cancelado	Substituído por NB4/NB5 (falhas induzidas).
A7 (Avaliação / Comparação)	Concluído	Métricas amostrais e CARE consolidados.
A8 (Análise / Interpretação)	Concluído	Discussão e ablação documentadas.
A9 (Relatórios)	Concluído	Relatório final entregue à FAPESP.

Tabela 7: Cronograma realizado vs. planejado (A1–A9 do plano FAPESP).

13 Matérias Cursadas na Faculdade

A Tabela 8 relaciona as disciplinas cursadas durante o período de vigência da IC e sua contribuição para o projeto.

14 Lições Aprendidas

(1) **Métricas importam mais que modelos** no cenário desbalanceado — a escolha de F_1 , AUC-ROC e CARE em vez de acurácia muda fundamentalmente a interpretação dos resultados. (2) **Pipeline antes de modelo** — bugs no pré-processamento (e.g., *leakage* do scaler ajustado em validação) podem esconder ou inflar resultados; a separação rígida treino/validação/teste e a fixação de *seeds* são pré-requisitos. (3) **Replicar antes de inovar** — a replicação do paper de Qi et al. (2024) (NB1) revelou limitações práticas (colapso do LSTM) que motivaram a migração para o paradigma não supervisionado. (4) **Hardware é restrição real** — a expectativa inicial de treinar tudo em CPU foi rapidamente abandonada; o uso de GPU passou de “conveniência” a pré-requisito. (5) **Anonimização de dados públicos** é uma limitação não discutida na maioria dos papers, mas que pode inviabilizar inteiras linhas de pesquisa — motivou o redirecionamento do escopo deste projeto.

Disciplina	Semestre	Relação com o projeto
Fenômenos Eletromagnéticos	2025.2	Base física para sensores SCADA (corrente, tensão, campo).
Prática em Projetos Extensionistas II	2025.2	Metodologia de projetos aplicados.
Programação Orientada a Objetos	2025.2	Modelagem dos pipelines de pré-processamento e modelos.
Lab. Sist. Computacionais: Circuitos Digitais	2025.2	Fundamentos de hardware digital.
Arquitetura e Organização de Computadores	2025.2	Compreensão da execução em GPU/TPU.
Circuitos Elétricos I	2025.2	Análise de variáveis elétricas das turbinas.
Projeto e Análise de Algoritmos	2025.2	Complexidade de algoritmos de seleção de <i>features</i> .
Análise de Sinais	2026.1	Tratamento de séries temporais SCADA, FFT, filtros.
Circuitos Elétricos II	2026.1	Análise de regimes transitórios e CA.
Introdução aos Materiais Elétricos	2026.1	Materiais de geradores e conversores.
Lab. Sist. Computacionais: Arq. e Org. de Computadores	2026.1	Prática de execução em hardware especializado.
Linguagens Formais e Autômatos	2026.1	Base teórica para máquinas de estado e parsing.
Algoritmos em Bioinformática	2026.1	Análise de sequências e distâncias — raciocínio algorítmico complementar ao projeto (disciplina obrigatória do currículo).
Fenômenos Eletromagnéticos Experimental	2026.1	Prática laboratorial complementar.

Tabela 8: Disciplinas cursadas durante a vigência da IC (2025.2 e 2026.1).

15 Trabalhos Futuros

Desdobramentos naturais deste trabalho incluem: (1) **Ensemble NB3 + NB5**, combinando o alto *recall* amostral do NB3 com a alta especificidade por evento do NB5, potencialmente atingindo um ponto operacional intermediário viável em produção. (2) **Focal Loss** no NB1 para mitigar o colapso do LSTM e melhorar o tratamento do desbalanceamento. (3) **Threshold por regime operacional** (partida, geração nominal, parada), reduzindo falsos positivos causados por variações legítimas. (4) **Generalização cross-farm** — avaliar os modelos do Wind Farm C nos Wind Farms A e B com técnicas de *domain adaptation*. (5) **Implementação de Transformer completo** (com *positional encoding* e blocos *encoder/decoder*) para fechar o item pendente do plano original.

16 Considerações Finais

Este relatório documenta os principais resultados obtidos na Iniciação Científica em detecção de anomalias em turbinas eólicas. Apesar da inviabilidade técnica de integrar dados meteorológicos exógenos — inicialmente o foco central do plano FAPESP —, o projeto se readequou produtivamente para a comparação entre paradigmas de detecção e a exploração de falhas induzidas como técnica de *data augmentation*. Os autoencoders não supervisionados se mostraram superiores aos modelos supervisionados sob desbalanceamento severo (AUC-ROC 0,87 vs. 0,66), e a calibração de *thresholds* via falhas sintéticas abriu uma linha de pesquisa promissora pela alta especificidade por evento que produz. A contribuição formativa para o aluno foi significativa, abrangendo desde os fundamentos de aprendizado profundo até pipelines de produção em PyTorch e Keras, otimização automática com Optuna e integração com frameworks de avaliação padronizados (CARE Score / EnergyFaultDetector). Todos os notebooks, experimentos, pipelines e resultados estão disponíveis publicamente no repositório GitHub do projeto: https://github.com/Thiago-Helene/IC_Thiago-Helene_Wind_Turbines.

Referências

- Haitham Afan, Ahmedbahaaaldin Ibrahim Ahmed Osman, Yusuf Essam, Al-Mahfoodh Ali Najah Ahmed, Yuk Huang, Ozgur Kisi, Mohsen Sherif, Ahmed Sefelnasr, Kwok Chau, and Ahmed El-Shafie. Modeling the fluctuations of groundwater level by employing ensemble deep learning techniques. *Engineering Applications of Computational Fluid Mechanics*, 15: 1420–1439, 09 2021. doi: 10.1080/19942060.2021.1974093.
- Shahabodin Afrasiabi, Mousa Afrasiabi, Benyamin Parang, Mohammad Mohammadi, Solmaz Kahourzade, and Amin Mahmoudi. Two-stage deep learning-based wind turbine condition monitoring using SCADA data. *IEEE Access*, 2020. doi: 10.1109/PEDES49360.2020.9379393.
- G V Ramesh Babu, Sagar Baburao Porlekar, H. Mohammed Ali, Veera Ankalu Vuyyuru, Mohammed Saleh Al Ansari, and I Infant Raj. Dynamic fault diagnosis in wind turbines: A GNN-LSTM approach, 2024.

- Espaciais (INPE). CPTEC/INPE - centro de previsão de tempo e estudos climáticos. Website. URL <https://www.cptec.inpe.br/>. Accessed: 2025-04-01.
- Subhajit Chatterjee and Yung-Cheol Byun. Leveraging generative adversarial networks for data augmentation to improve fault detection in wind turbines with imbalanced data. *Results in Engineering*, 25:103991, 2025. doi: 10.1016/j.rineng.2025.103991.
- Kukjin Choi, Jihun Yi, Changhwa Park, and Sungroh Yoon. Deep learning for anomaly detection in time-series data: Review, analysis, and guidelines. *IEEE Access*, 9, 2021. doi: 10.1109/ACCESS.2021.3107975.
- C. J. Crabtree, D. Zappalà, and P. J. Tavner. Survey of commercially available condition monitoring systems for wind turbines. Technical report for supergen wind energy technologies consortium, Durham University, School of Engineering and Computing Sciences, 2011. URL <https://durham-repository.worktribe.com/OutputFile/1632372>.
- Ali Fazli and Javad Poshtan. Wind turbine fault detection and isolation robust against data imbalance using KNN. *IET Renewable Power Generation*, 2024. doi: 10.1002/ese3.1706. URL <https://scijournals.onlinelibrary.wiley.com/doi/10.1002/ese3.1706>.
- Bisrat Gebreamlak. Hybrid CNN-integrated LSTM for fault detection and diagnosis of wind turbines. Master's thesis, School of Computing, University of Eastern Finland, May 2024. URL <https://erepo.uef.fi/bitstreams/3147532c-1214-4c24-9517-b7b04cab97db/download>. Accessed: 2025-04-05.
- E Gonzalez, Bruce Stephen, David Infield, and Julio J. Melero. On the use of high-frequency SCADA data for improved wind turbine performance monitoring. *Journal of Physics*, 2017. doi: 10.1088/1742-6596/926/1/012009.
- J.Y. Guo, Z.Y. Wang, H. Li, and C. Guedes Soares. A transformer-based fault detection framework for offshore wind turbines based on SCADA data. In *Advances in Maritime Technology and Engineering*. CRC Press / Taylor & Francis, Boca Raton, FL, 2024. doi: 10.1201/9781003508762-66. URL <https://www.taylorfrancis.com/chapters/edit/10.1201/9781003508762-66/transformer-based-fault-detection-framework-offshore-wind-turbines-based-scada-data-guo-wang-li-guedes-soares>. Accessed: 2025-04-05.

- Markus Kreutz, Abderrahim Ait Alla, Kamaloddin Varasteh, Stephan Oelker, Andreas Greulich, Michael Freitag, and Klaus-Dieter Thoben. Machine learning-based icing prediction on wind turbines. *Procedia CIRP*, 2019. doi: 10.1016/j.procir.2019.03.073.
- Wen-Hui Lin, Ping Wang, Kuo-Ming Chao, Hsiao-Chung Lin, Zongyu Yang, and Yu-Huang Lai. Wind power forecasting with deep learning networks: Time-series forecasting. *Applied Sciences*, 2021. doi: 10.3390/app112110335.
- Junyan Ma and Yiping Yuan. Application of SCADA data in wind turbine fault detection – a review. *Sensor Review*, 42(6):509–525, 2022. doi: 10.1108/SR-06-2022-0255. URL <https://www.emerald.com/insight/content/doi/10.1108/sr-06-2022-0255/full/html>.
- P. M. Muralidharan, Gaurav Thakur, M Shalini, Vikalp Sharma, Abootharmahmoodshakir, and Anishkumar Dhablia. A review on condition monitoring of wind turbines using machine learning techniques. In *E3S Web of Conferences*, volume 540, page 03003, 2024. doi: 10.1051/e3sconf/202454003003. URL https://www.e3s-conferences.org/articles/e3sconf/abs/2024/70/e3sconf_icpes2024_03003/e3sconf_icpes2024_03003.html.
- Eddie Yin Kwee Ng and Jian Tiong Lim. Machine learning on fault diagnosis in wind turbines. *Fluids*, 7(12):371, 2022. doi: 10.3390/fluids7120371. URL <https://www.mdpi.com/2311-5521/7/12/371>.
- Adaiton Oliveira-Filho, Monelle Comeau, James Cave, Charbel Nasr, Pascal Côté, and Antoine Tahan. Wind turbine SCADA data imbalance: A review of its impact on health condition analyses and mitigation strategies. *Energies*, 2024. doi: 10.3390/en18010059. URL <https://www.mdpi.com/1996-1073/18/1/59>.
- OpenWeatherMap. Weather API. Website API Documentation. URL <https://openweathermap.org/api>. Accessed: 2025-04-01.
- Van Phung and Eun Rhee. A high-accuracy model average ensemble of convolutional neural networks for classification of cloud image patches on small datasets. *Applied Sciences*, 9: 4500, 10 2019. doi: 10.3390/app9214500.
- Lin Qi, Qianqian Zhang, Yunjie Xie, Jian Zhang, and Jinran Ke. Research on wind turbine fault detection based on cnn-lstm. *Energies*, 17(17):4497, 2024. doi: 10.3390/en17174497.

- Abirami Sasinthiran, G. Sakthivel, and Ramesh Ragala. A review of artificial intelligence applications in wind turbine health monitoring. *Wind Engineering*, 2024. doi: 10.1080/14786451.2024.2326296.
- Meik Schlechtingen, Ilmar F. Santos, and Sofiane Achiche. Wind turbine condition monitoring based on SCADA data using normal behavior models. *Applied Soft Computing*, 13(1):259–270, 2013. doi: 10.1016/j.asoc.2012.08.033. URL <https://www.sciencedirect.com/science/article/abs/pii/S1568494612003821?via%3Dihub>.
- sltzgs. Wind_Turbine_SCADA_open_data: A collection of open data for wind turbine scada systems. GitHub repository. URL https://github.com/sltzgs/Wind_Turbine_SCADA_open_data. Accessed: 2025-04-01.
- Abhinav Tummala, Ratna Kishore Velamati, Dheeraj Kumar Sinha, Vankara Indraja, and V. Shekhar. A review on small scale wind turbines. *Renewable and Sustainable Energy Reviews*, 56:1351–1371, 2016. doi: 10.1016/j.rser.2015.12.027.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, Long Beach, CA, 2017. URL <https://arxiv.org/abs/1706.03762>.
- Hong Wang, Taikun Li, Mingyang Xie, Wenfang Tian, and Wei Han. Wind turbine fault diagnosis with imbalanced scada data using generative adversarial networks. *Energies*, 18(5):1158, 2025. doi: 10.3390/en18051158.
- Cheng Xiao, Zuojun Liu, Tieling Zhang, and Xu Zhang. Deep learning method for fault detection of wind turbine converter. *Applied Sciences*, 11(3):1280, 2021. doi: 10.3390/app11031280. URL <https://www.mdpi.com/2076-3417/11/3/1280>.